UNIVERSIDAD DE MÁLAGA

E.T.S. INGENIERÍA INFORMÁTICA
UNIVERSIDAD DE MÁLAGA

Graduado en Ingeniería de Computadores

PioTGuard - Sistema de Seguridad con Videovigilancia
PioTGuard – Security System with Videosurveillance

Realizado por
David Ruiz Luque

Tutorizado por
Vicente Jesús Benjumea García

Departamento
Lenguajes y Ciencias de la Computación

MÁLAGA, junio de 2024

UNIVERSIDAD DE MÁLAGA

E.T.S. INGENIERÍA INFORMÁTICA
UNIVERSIDAD DE MÁLAGA

ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA
GRADUADO EN INGENIERÍA DE COMPUTADORES

PIOTGUARD – SISTEMA DE SEGURIDAD CON VIDEOVIGILANCIA

PIOTGUARD – SECURITY SYSTEM WITH VIDEOSURVEILLANCE

Realizado por
David Ruiz Luque

Tutorizado por
Vicente Jesús Benjumea García

Departamento
Lenguajes y Ciencias de la Computación

UNIVERSIDAD DE MÁLAGA
MÁLAGA, JUNIO DE 2024

Fecha defensa: julio de 2024

Abstract

This Final Degree Project consists of the design and implementation of a surveillance system, monitoring the area with different sensors and a camera, offering real-time video capture (streaming), and video recording in case an anomaly is detected. The application domain focuses on security and surveillance, specifically in restaurants, considering protection not only against external threats, but also against internal risks.

The system is composed of an ESP-WROOM-32, which constantly monitors and sends the information obtained from its three sensors to the server: the presence sensor, which detects movement in specific areas; a magnetic sensor, placed on doors and windows to control their opening; and the gas sensor, which measures the levels of gas or smoke in the environment, essential for the early detection of fires or leaks.

The core of the system, and the server, is a Raspberry Pi with a camera, offering streaming, and capturing video in response to the activation of any of the aforementioned sensors. This functionality is key to documenting and managing security incidents effectively.

And finally, we have an Android application, with which we can configure the system and the sensors, as well as being able to see the history of events that the system has recorded (anomalies that have triggered the alarm), with their associated recording. The application will also notify us if an anomaly is detected.

All system connections will use SSL/TLS, encrypting all information sent between the different devices. In addition, both (client and server), will be authenticated using mTLS (Mutual TLS), ensuring that both the client and server authenticate each other before establishing the connection.

With this we get a reliable, secure and functional system, being able to see at all times the streaming provided by the server, and ensuring that the system will warn us and record at all times if any anomaly is detected.

Keywords: Raspberry Pi, ESP32, Android, MQTT, Apache, SSL, TLS, MTLS, Streaming, MariaDB, IoT, FreeRTOS, GStreamer.

Resumen

Este Trabajo de Fin de Grado consiste en el diseño y realización de un sistema de vigilancia, monitorizando el área donde se instale con diferentes sensores y una cámara, ofreciendo captura de vídeo en tiempo real (streaming), y grabación de vídeo en caso de detectarse una anomalía. El dominio de aplicación se centra en la seguridad y vigilancia, específicamente en restaurantes, considerando la protección no solo contra amenazas externas, sino también contra riesgos internos.

El sistema está compuesto de un ESP-WROOM-32, el cual monitoriza y envía constantemente la información obtenida al servidor de sus tres sensores: el sensor de presencia, que detecta movimiento en áreas específicas; un sensor magnético, colocado en puertas y ventanas para controlar su apertura; y el sensor de gas, que mide los niveles de gas o humo en el ambiente, esencial para la detección temprana de incendios o fugas.

El núcleo del sistema y el servidor, es una Raspberry Pi con una cámara, ofreciendo streaming, y captura de vídeo en respuesta a la activación de cualquiera de los sensores mencionados. Esta funcionalidad es clave para documentar y manejar los incidentes de seguridad eficazmente.

Y por último, tenemos una aplicación Android, con la cual podremos configurar el sistema y los sensores, además de poder ver el historial de eventos registrado (anomalías que han disparado la alarma), con su grabación asociada. La aplicación también nos notificará en caso de detectarse una anomalía.

Todas las conexiones del sistema utilizarán SSL/TLS, cifrando toda la información enviada entre los diferentes dispositivos. Además, ambos (clientes y servidor), se autenticarán mediante mTLS (Mutual TLS), asegurando que tanto el cliente como el servidor se autentiquen mutuamente antes de establecer la conexión.

Con esto conseguimos un sistema fiable, seguro, y funcional, pudiendo ver en todo momento el streaming que proporciona el servidor, y asegurándonos que el sistema nos avisará y grabará en todo momento si se detecta alguna anomalía.

Palabras clave: **Raspberry Pi, ESP32, Android, MQTT, Apache, SSL, TLS, MTLS, Streaming, MariaDB, IoT, FreeRTOS, GStreamer.**

Índice

1. Introducción **11**

1.1. Motivación . . . 11

1.2. Objetivos . . . 11

1.3. Estructura del documento . . . 12

2. Antecedentes y estado actual del problema **15**

3. Tecnologías y Herramientas Utilizadas **17**

4. Metodología de trabajo. Análisis y requisitos **25**

4.1. Metodología de trabajo . . . 25

4.2. Análisis y diseño de requisitos . . . 27

4.2.1. Requisitos funcionales . . . 27

4.2.2. Requisitos no funcionales . . . 28

5. Diseño funcional del sistema **31**

5.1. Diagrama General o de Servicios . . . 31

5.2. Diagrama de Secuencia . . . 33

5.2.1. Inicio de Sesión, Manejo de Cuentas . . . 33

5.2.2. Funcionalidades de la App Android . . . 34

5.2.3. Detección de una anomalía . . . 35

5.3. Diagrama de Casos de Uso . . . 36

6. Diseño físico del sistema **39**

6.1. Conexionado ESP-WROOM-32 . . . 40

6.2. Conexionado Raspberry Pi 3B+ . . . 43

6.3. Sistema final . . . 44

7. Certificados SSL/TLS y autenticación mTLS **45**

7.1. Principios de SSL/TLS . . . 46

7.2. Mutual TLS . . . 47
7.3. Funcionamiento de SSL/TLS y mTLS . . . 47
7.4. Utilidad y Necesidad de mTLS . . . 49
7.5. Contenido de los certificados . . . 50
7.5.1. Contenido de los certificados de las CAs . . . 50
7.5.2. Contenido de los certificados del servidor . . . 51
7.5.3. Certificados de los Clientes . . . 51
7.6. Generación de certificados . . . 53

8. Preparación del Servidor **55**
8.1. UFW . . . 55
8.2. Mosquitto . . . 56
8.3. GStreamer . . . 56
8.4. MariaDB . . . 58
8.4.1. Configuración . . . 58
8.4.2. Usuarios . . . 58
8.4.3. Base de datos y tablas . . . 59
8.5. Apache . . . 59
8.6. Servicios y systemd . . . 60
8.7. NO-IP . . . 61

9. Programación del ESP32 y sensores **63**
9.1. Modos de operación . . . 63
9.2. FreeRTOS . . . 64
9.3. Publicación de topics. Función publishValues . . . 65
9.4. Manejo de topics. Función topicHandler . . . 65
9.5. Configuración de los pines de los sensores . . . 65
9.6. Otras funciones . . . 66

10. Programación en la Raspberry Pi **67**
10.1. Servidor RTSP. GStreamer . . . 67
10.1.1. Configuración del Servidor RTSP . . . 67

10.1.2. Autenticación y SSL/TLS . . . 68
10.1.3. Limpieza de Sesiones. Función timeout . . . 68
10.1.4. Ejecución del Bucle Principal . . . 69
10.2. Cliente servidor . . . 69
10.2.1. Conexión y Configuración del cliente MariaDB . . . 69
10.2.2. Grabación de Vídeo. Función record_video . . . 70
10.2.3. Función Principal, main . . . 71

11. Programación de la App Android **73**
11.1. Estructura general . . . 73
11.1.1. Actividades (Activities) . . . 73
11.1.2. Servicios (Services) . . . 73
11.2. Funcionalidades Principales . . . 74
11.2.1. Gestión del Sistema y Sensores . . . 74
11.2.2. Streaming y GStreamer . . . 74
11.2.3. Gestión de Credenciales . . . 76
11.2.4. Historial de Eventos y Grabaciones de Vídeo . . . 77

12. Resultados y Líneas Futuras **79**
12.1. Resultados . . . 79
12.2. Conclusiones . . . 80
12.3. Trabajos Futuros . . . 81

13. Referencias Bibliográficas **83**

Apéndice A. Contenido de archivos **87**
A.1. Archivo de configuración **openssl.cnf** . . . 87
A.2. Archivo de configuración Mosquitto **conf.cnf** . . . 88
A.3. Archivo de configuración MariaDB **50-server.cnf** . . . 88
A.4. Archivo de configuración MariaDB **50-clients.cnf** . . . 89
A.5. Archivo de configuración Apache **default-ssl.cnf** . . . 90

Índice de figuras

1. Ejemplo de Tareas del Proyecto en Trello. 26
2. Esquema General de Servicios 32
3. Diagrama de Secuencia Usuarios 34
4. Diagrama de Secuencia Funcionalidades 35
5. Diagrama de Secuencia Anomalía 36
6. Diagrama de Casos de Uso 37
7. Pinout ESP-WROOM-32 40
8. Conexionado ESP-WROOM-32 41
9. Pinout Sensor de Gas MQ2 42
10. Pinout Sensor de Presencia PIR HC-SR501 42
11. Interruptor Magnético 43
12. Puertos Raspberry Pi 3B+ 43
13. Conexionado Raspberry Pi 3B+ 43
14. Cámara IR-Cut 1080p y LEDs IR 43
15. Esquema Físico General 44
16. Funcionamiento de una conexión SSL/TLS 49
17. Ejemplo de un certificado del cliente 52

1 Introducción

1.1. Motivación

La motivación de este proyecto surge de la necesidad de mantener un entorno seguro y protegido en el sector de la hostelería, especialmente en los restaurantes, donde constantemente, se enfrentan desde amenazas tales como intrusos y robos, hasta incendios. Dado que este sector está siempre abierto al público, es importante contar con un sistema de seguridad eficiente ofreciendo una supervisión en tiempo real. A este motivo se le suma la seguridad e integridad de las conexiones y datos compartidos entre las conexiones del sistema.

1.2. Objetivos

Este proyecto tiene como objetivo diseñar e implementar un sistema de seguridad utilizando dispositivos IoT y sistemas embebidos. Se desarrollará un sistema que utiliza diversos sensores para detectar movimiento y la apertura de puertas y ventanas en respuesta a amenazas externas, así como la presencia de gas y humo ante amenazas internas. Incluirá también una cámara que iniciará una grabación, si los sensores descritos detectan alguna anomalía, además de ofrecer captura de vídeo en tiempo real (streaming) en cualquier momento. Con la aplicación Android, se podrá configurar el sistema, y alertará de inmediato sobre cualquier incidente. Además, todas las conexiones del sistema usarán SSL/TLS, y mTLS para la autenticación mutua entre dispositivos, obteniendo un sistema donde todas las comunicaciones son seguras y resistentes a ataques.

En general, este proyecto busca implementar, en el ámbito de los sistemas embebidos e IoT, una solución que alcanza el nivel de seguridad de sistemas más complejos en el sector de la seguridad de establecimientos públicos, garantizando la protección de clientes, personal e infraestructura en este sector.

1.3. Estructura del documento

En el capítulo de **Introducción** (ver sección 1) , se describen los puntos fundamentales y objetivos de este ”trabajo fin de estudios”. También se describe la estructura de este documento.

En el capítulo de **Antecedentes y Estado Actual del Problema** (ver sección 2), se describe la necesidad de reforzar la seguridad en los restaurantes ante amenazas tanto externas como internas, incluyendo robos, incendios y fugas de gas. También se explica en qué situación se encuentran este tipo sistemas y aplicaciones que solucionen el problema, y cómo se podría utilizar e implementar el sistema resultado de este proyecto.

En el capítulo de las **Tecnologías y Herramientas Utilizadas** (ver sección 3), se describe qué tecnologías han permitido desarrollar este proyecto, incluyendo no sólo los dispositivos físicos, también las librerías, programas y marcos de trabajo utilizados.

En el capítulo sobre la **Metodología de Trabajo**, junto con el **Análisis y Requisitos** (ver sección 4), y **Diseño Funcional del Sistema** (ver sección 5), se detallará la metodología de trabajo seguida en el desarrollo de este proyecto, y se expondrán los requisitos funcionales y no funcionales. Además, se mostrarán y explicarán diferentes diagramas como casos de uso, diagramas de secuencia y esquemas de funcionalidades, proporcionando una visión clara de cómo se usa y funciona el sistema.

En la sección **Diseño Físico del Sistema** (ver sección 6), se describe cómo está estructurado físicamente, incluyendo figuras sobre el conexionado del ESP32 y la integración de las demás partes del sistema. También se consideran las posibles ubicaciones de los dispositivos dentro del restaurante.

En la sección **Preparación del Entorno y Servidor en la Raspberry Pi** (ver sección 8), explica en detalle el proceso de instalación, configuración y preparación de la Raspberry Pi para que actúe como servidor de los diferentes servicios esenciales del sistema, asegurándose de que todos estos servicios siempre estén disponibles.

En el capítulo **Certificados SSL/TLS y Autenticación mTLS** (ver sección 7), se detallará como ha sido el proceso generación, configuración y uso de certificados para asegurar el cifrado de las conexiones entre los dispositivos del sistema, y la autenticación mutua entre clientes y servidor.

En la sección **Programación del ESP32 y Sensores** (ver sección 9), se presenta cómo se ha desarrollado la programación de los diferentes sensores y el propio ESP32, permitiendo la monitorización en paralelo de los sensores y la transmisión de la información obtenida de estos al servidor.

En el capítulo **Programación del Cliente en la Raspberry Pi** (ver sección 10), tras haber realizado la preparación del entorno, se detalla cómo se ha diseñado el cliente dentro del servidor, que se usará para la inserción de información en la base de datos, grabación de vídeo, y activación y desactivación del streaming.

En la sección **Programación de la App Android** (ver sección 11), que representa el último punto en el desarrollo del sistema, se detalla cómo se ha afrontado el diseño y programación de la App, la cual nos avisará si se detectan anomalías, además de permitir ver el streaming en tiempo real y mostrarnos información sobre el incidente detectado y su grabación asociada.

En el último capítulo, **Resultados y Líneas Futuras** (ver sección **??**), se podrá observar el funcionamiento del sistema al completo, describiendo todas las funcionalidades disponibles. Se expondrán las conclusiones y líneas futuras, proponiendo posibles mejoras y trabajos futuros.

2

Antecedentes y estado actual del problema

La seguridad en el sector de la hostelería, y especialmente en restaurantes, siempre ha sido una preocupación constante debido a las amenazas externas, desde robos, intrusos y vandalismo, hasta amenazas internas, como pueden ser fugas de gas o la importancia de detectar y evitar incendios. Los métodos tradicionales de seguridad han evolucionado hacia sistemas avanzados y complejos que utilizan tecnología punta y emergente para abordar y solucionar estos riesgos de manera efectiva. Con la llegada de estas tecnologías, la industria obtiene una mejora más que significativa en la capacidad de detección y respuesta a estos incidentes. Las tecnologías modernas utilizadas en estos sistemas permiten una integración profunda y una respuesta rápida, asegurando la prevención de amenazas.

En la actualidad, existe una amplia variedad de sistemas y aplicaciones que han demostrado ser efectivos en la seguridad de este sector. Estos sistemas incluyen cámaras de seguridad avanzadas, sensores de movimiento y de todo tipo, y sistemas de alarma que ofrecen una monitorización dedicada, obteniendo una cobertura completa y una respuesta rápida a amenazas e incidentes.

A pesar de todos los avances, el desafío aún sigue en optimizar y aumentar aún más la seguridad, especialmente en la protección de conexiones y comunicaciones dentro del sistema instalado. La seguridad de las conexiones en cualquier sistema es crítica para prevenir ciberataques que puedan comprometer la integridad y disponibilidad de datos, y en este caso, la propia seguridad del establecimiento donde se instale el sistema. A medida que más dispositivos se interconectan, la necesidad de asegurar estas conexiones se vuelve aún más importante.

Este proyecto no busca reemplazar los sistemas de seguridad existentes en el sector, sino complementarlos aprovechando IoT y los sistemas embebidos, implementando autenticación y seguridad en sus conexiones y comunicaciones que, como resultado, se obtendrá un sistema potente, seguro y eficiente utilizando dispositivos económicos y fiables.

El objetivo es implementar un sistema de seguridad, que con el uso de varios sensores, capture movimiento y la apertura de puertas y ventanas para responder a amenazas externas, así como detectar gas y humo en caso de amenazas internas. Además, el sistema incluye el uso de una cámara que iniciará la grabación ante cualquier anomalía detectada y proporcionará streaming en tiempo real, avisando la App Android de que ha ocurrido algún incidente. El sistema reforzará la seguridad mediante el cifrado y la autenticación mutua en todas sus conexiones. Esta implementación busca ofrecer una protección completa, tanto en el establecimiento donde se instale, como en las propias conexiones y datos utilizados en este sistema, mejorando la seguridad en general.

El sistema propuesto integra un sensor de presencia (movimiento), uno magnético (puertas y ventanas) y otro de gas conectados a un ESP-WROOM-32, y una Raspberry Pi que con su cámara ofrecerá streaming en tiempo real, y grabación de vídeo en respuesta a anomalías detectadas. La App Android facilitará de una manera visual el control remoto del sistema, configurando los sensores y su monitorización, viendo las grabaciones realizadas y el streaming ofrecido, con notificaciones inmediatas ante cualquier incidente.

Implementando SSL/TLS y mTLS para la autenticación mutua entre dispositivos, el sistema garantiza que todas las comunicaciones sean seguras y estén protegidas contra ataques, verificando la identidad tanto del servidor como de los clientes. Este enfoque refuerza la seguridad tanto física como de datos, asegurando su integridad y disponibilidad en todo momento.

3

Tecnologías y Herramientas Utilizadas

MQTT

MQTT (Message Queuing Telemetry Transport) es un protocolo de mensajería ligero y eficiente, diseñado para dispositivos de baja potencia y redes con un ancho de banda limitado. Es ideal para la comunicación entre aplicaciones de Internet de las Cosas (IoT) debido a su diseño simple y su capacidad para establecer comunicaciones fiables y seguras con un bajo presupuesto. Funciona sobre Internet utilizando el protocolo TCP/IP, permitiendo su uso sobre WiFi o Ethernet.

A diferencia de MQTT, existen otros protocolos como Zigbee y Z-Wave (radio), Bluetooth (corto alcance), y LoRa (larga distancia) que se utilizan en IoT según las necesidades de alcance, energía y datos necesitados.

MQTT se basa en un modelo de publicación y suscripción:

- **Topics (Temas)**: En MQTT, los mensajes se organizan en temas (topics), que son canales de comunicación. Por ejemplo, ”sensors/temperature”podría ser un topic para enviar mensajes relacionados con la temperatura.
- **Publicar (Publish)**: Un dispositivo (cliente), puede enviar un mensaje a un topic específico al servidor. Esto se llama publish. Por ejemplo, un sensor de temperatura puede publicar sus lecturas en el topic ”sensors/temperature”.
- **Suscribir (Subscribe)**: Otros dispositivos pueden suscribirse a un topic para recibir los mensajes. Cuando un cliente se suscribe a un topic, el broker MQTT le envía todos los mensajes que se publiquen en ese topic.

- **Broker (Intermediario)**: El broker MQTT actúa como intermediario. Recibe los mensajes de los clientes que publican en un topic y los envía a todos los clientes suscritos a ese topic. Además, gestiona las conexiones, se encarga de la autenticación y la autorización, y asegura que la calidad del servicio (QoS) sea la adecuada.

Todos los mensajes enviados por MQTT serán en **formato JSON**, teniendo una estructura de datos clara tanto para el servidor como para los clientes suscritos. Este formato facilita la inclusión de múltiples tipos de datos en un solo mensaje, como por ejemplo, el tipo de sensor, la hora de la detección y el valor de la lectura del sensor.

GStreamer

GStreamer es un marco de trabajo multimedia potente y versátil, principalmente utilizado para desarrollar aplicaciones de procesamiento de audio y vídeo. Es de código abierto y está disponible bajo la licencia LGPL, lo que permite su uso en una amplia variedad de aplicaciones.

GStreamer actúa como un pipeline de procesamiento de datos, donde estos fluyen a través de una serie de elementos que pueden generarlos, manipularlos o consumirlos. Estos elementos se enlazan en una cadena que puede ser tan simple o tan compleja como necesite la aplicación. Cada elemento del pipeline está diseñado para realizar una tarea específica, como decodificar un archivo de vídeo, aplicar efectos, codificar en un formato diferente, o transmitir el vídeo a través de la red.

RTSP

RTSP (Real-Time Streaming Protocol) es un protocolo de red utilizado para controlar la transmisión de datos multimedia en tiempo real. Específicamente diseñado para la entrega de medios, RTSP permite a los clientes emitir comandos como “play”, “pause” y “stop” a los servidores de streaming.

En este proyecto se hará streaming mediante RTSP con GStreamer, lo que permitirá capturar, procesar y transmitir video en tiempo real de manera eficiente y flexible.

MariaDB

MariaDB es un sistema de gestión de bases de datos relacional de código abierto que surgió como un fork de MySQL. Es completamente compatible con MySQL, permitiendo una fácil migración de sistemas existentes sin necesidad de cambiar las aplicaciones. Ampliamente adoptado en la comunidad de código abierto, MariaDB ofrece una solución robusta y eficiente para la gestión de datos, siendo una elección popular en muchas aplicaciones y sistemas operativos debido a su licencia GPL que promueve la libertad de uso y distribución.

SSL/TLS

SSL (Secure Sockets Layer) y TLS (Transport Layer Security) son protocolos de cifrado diseñados para asegurar las comunicaciones a través de redes informáticas. Aunque TLS es el sucesor directo de SSL y es más utilizado en la actualidad, el término ”SSL”todavía es comúnmente usado para referirse a ambos.

mTLS

mTLS, o Mutual TLS, es una variante de este protocolo donde el cliente y el servidor se autentican mutuamente mediante certificados, antes de establecer una conexión. Este método es más seguro que el TLS estándar, donde solamente se autentica el servidor. mTLS es necesario en entornos donde la seguridad en las comunicaciones es crítica, necesitando que ambos lados de la conexión necesiten confirmar la identidad del otro, como sucede en las comunicaciones entre microservicios en arquitecturas de software avanzadas.

En este sistema en concreto, es muy importante aplicar estos protocolos en las comunicaciones para garantizar la confidencialidad e integridad de los datos transmitidos entre el dispositivo Android, el servidor Raspberry Pi, y el ESP32.

En el capítulo **Certificados SSL/TLS y autenticación mTLS** (ver capítulo 7), se explicará en detalle el funcionamiento y características de estos dos protocolos.

Raspberry Pi Model 3B+ y cámara

La Raspberry Pi es un pequeño ordenador de bajo costo y muy versátil, utilizado hoy en día para una amplia variedad de propósitos, desde la educación en ciencias de la computación, hasta automatización del hogar y prototipos industriales.

En este proyecto, la Raspberry Pi 3B+ está equipada con una cámara de alta definición (1080p) que incluye un filtro de infrarrojos (IR). Esta capacidad le permite a la cámara capturar imágenes claras durante el día y también obtener imágenes detalladas en condiciones de baja luz o en la oscuridad, lo cual es perfecto para aplicaciones de vigilancia y seguridad.

Raspberry Pi OS

Raspberry Pi OS es un sistema operativo gratuito y de código abierto basado en Debian, creado especialmente para el hardware de Raspberry Pi. Esta versión de Linux está diseñada para mejorar el rendimiento y la eficiencia de las Raspberry Pi, que son pequeñas computadoras muy utilizadas en proyectos educativos, de investigación y desarrollo, así como por aficionados y profesionales en diversas aplicaciones.

OpenSSL

OpenSSL es una herramienta de software de código abierto que se utiliza para implementar protocolos de seguridad, como SSL (Secure Sockets Layer) y TLS (Transport Layer Security). Estas tecnologías permiten cifrar las comunicaciones en redes. OpenSSL se utiliza para generar certificados digitales, gestionar claves criptográficas y establecer conexiones seguras en aplicaciones web y otros servicios de red.

UFW

UFW (Uncomplicated Firewall) es una herramienta para gestionar las reglas del cortafuegos en sistemas operativos basados en Linux. Su propósito es simplificar la configuración del firewall, permitiendo a los usuarios permitir o bloquear conexiones de red con comandos sencillos.

Apache

Apache, también conocido como Apache HTTP Server, es un servidor web de código abierto. Es uno de los servidores web más populares y ampliamente utilizados en Internet. Apache se destaca por su robustez, flexibilidad y rendimiento, lo que lo hace adecuado para todo, desde pequeños sitios de web personales hasta grandes portales de comercio electrónico.

Watchdog

Un watchdog es un sistema de supervisión que se utiliza para detectar y recuperar fallos en un sistema informático. Se trata de un mecanismo de seguridad que reinicia un sistema o un componente del sistema si detecta que no está funcionando correctamente o se ha caído. Los watchdogs son esenciales en aplicaciones críticas donde es necesario garantizar la continuidad del servicio y minimizar el tiempo de inactividad.

Systemd

Systemd es un sistema de inicialización y gestión de servicios para sistemas operativos basados en Linux. Es uno de los componentes más críticos del sistema, ya que se ejecuta como el primer proceso durante el arranque y administra todos los procesos y servicios del sistema a lo largo de su ejecución.

Se utilizará systemd como watchdog para los servicios del sistema, garantizando que si algún servicio falla o cae, systemd tomará las medidas necesarias para reiniciarlo y mantener el sistema en funcionamiento.

NO-IP

NO-IP es un proveedor de servicios de DNS dinámico (DDNS) que permite a los usuarios asignar un nombre de dominio fijo a una dirección IP dinámica. Este servicio es especialmente útil para personas que necesitan acceder a sus dispositivos de red, como cámaras de seguridad, sistemas de gestión de redes domésticas, servidores personales y otros dispositivos conectados, desde ubicaciones remotas a través de Internet.

ESP-WROOM-32 y sensores

El ESP-WROOM-32 es un potente módulo basado en el chip ESP32 de Espressif, que ofrece una solución ideal para gran variedad de aplicaciones IoT. Este módulo destaca por soportar Wi-Fi y Bluetooth, siendo el dispositivo perfecto para proyectos que necesitan conectividad inalámbrica y bajo consumo de energía.

En este proyecto, el ESP-WROOM-32 se utiliza junto a tres sensores:

1. **Sensor PIR HC-SR501**: Sensor de movimiento que utiliza infrarrojos para detectar cambios en los niveles de calor en el ambiente, perfecto para identificar la presencia de personas o animales. Es ideal para sistemas de seguridad y automatización del hogar donde la detección de movimiento es crucial.

2. **Sensor de gas MQ-2**: Este sensor es capaz de detectar gases combustibles y humo. El MQ-2 es utilizado comúnmente en sistemas de alarma para la detección temprana de posibles incendios o fugas de gas.

3. **Sensor Magnético**: Este tipo de sensor se utilizará para detectar la apertura o cierre de puertas y ventanas.

FreeRTOS

FreeRTOS es un sistema operativo en tiempo real (RTOS) de código abierto, diseñado para ejecutarse en microcontroladores y dispositivos embebidos de pequeña escala. Es ampliamente utilizado en aplicaciones donde se requiere una gestión eficiente y precisa del tiempo para el control de hardware y la ejecución de tareas. FreeRTOS facilita la programación de aplicaciones complejas en dispositivos con recursos limitados, ofreciendo un control riguroso sobre la funcionalidad de multitarea.

Android

Android es un sistema operativo basado en el kernel de Linux, diseñado principalmente para dispositivos móviles con pantalla táctil, como teléfonos inteligentes y tabletas. Además, Android se ha expandido para funcionar en una amplia gama de dispositivos, incluyendo relojes inteligentes (a través de Wear OS), televisores (Android TV), automóviles (Android Auto) y muchos más.

Dispositivo Android

Los dispositivos Android son dispositivos electrónicos, como teléfonos inteligentes y tabletas, que utilizan el sistema operativo Android. Estos dispositivos son utilizados para una amplia gama de aplicaciones.

En este proyecto, utilizaremos un smartphone Android para la gestión y control del sistema de seguridad, recibiendo notificaciones en caso de que el sistema detecte algún incidente.

Lenguajes de Programación

En el desarrollo de este sistema, se han utilizado varios lenguajes de programación, cada uno ejerciendo un papel fundamental en la implementación de los diferentes componentes del proyecto:

- **C**: Este lenguaje se ha utilizado principalmente para el desarrollo de las aplicaciones que corren en el ESP32 y en la Raspberry Pi. En el ESP32, el código en C maneja la lógica de los sensores, la conexión WiFi, y la comunicación con el servidor MQTT utilizando el protocolo mTLS para proteger las comunicaciones. En la Raspberry Pi, C se ha utilizado para implementar el servidor RTSP con GStreamer, encargado de transmitir el vídeo en tiempo real. En cuando a la App Android, C se ha utilizado en el desarrollo del cliente RTSP con GStreamer.

- **Kotlin**: Este lenguaje se ha utilizado para desarrollar la App Android. Kotlin es utilizado para manejar la lógica de la aplicación, las conexiones con el servidor MQTT, y para gestionar el sistema de seguridad al completo.

- **Java**: Aunque Kotlin es el lenguaje principal para la aplicación Android, se ha utilizado Java para algunas implementaciones que son críticas o necesitan integrarse con bibliotecas que originalmente están escritas en Java. Además, algunas funciones críticas del manejo de GStreamer en Android están implementadas en C y se comunican con Kotlin a través de JNI (Java Native Interface), lo que requiere una comprensión de profunda de Java para la integración.

- **Bash**: Se han utilizado scripts en Bash para tareas de automatización en el entorno del servidor, tales como la configuración inicial del sistema, la gestión de certificados y la configuración de servicios de red. Bash también se ha utilizado en scripts destinados al despliegue y mantenimiento de los servicios en la Raspberry Pi.

GitHub

GitHub es una plataforma de desarrollo colaborativo y control de versiones basada en Git. Permite a los desarrolladores gestionar y almacenar su código de manera eficiente, realizar seguimiento de cambios, colaborar con otros, y manejar versiones del software. GitHub facilita la revisión de código, la gestión de pull requests, y la resolución de conflictos, asegurando un flujo de trabajo organizado y coordinado en proyectos de desarrollo de software.

Trello

Para la planificación y gestión del trabajo, se ha utilizado Trello. Trello es una web de gestión de proyectos que utiliza tableros, listas y tarjetas para organizar tareas y proyectos. Facilita la planificación, asignación y seguimiento del trabajo de manera visual, permitiendo una gestión eficiente y colaborativa de proyectos.

4

Metodología de trabajo. Análisis y requisitos

4.1. Metodología de trabajo

Este proyecto se ha realizado siguiendo una metodología de desarrollo Ágil, específicamente Scrum. **Scrum**, es un marco de trabajo que fomenta la colaboración entre equipos y el desarrollo iterativo, permitiendo adaptarse rápidamente a los cambios y necesidades del cliente.

El trabajo se organizó en ciclos cortos llamados **Sprints**. La duración de cada Sprint ha sido de unas dos semanas aproximadamente. Al final de cada Sprint, se realizaban reuniones con el cliente para presentar el progreso del proyecto, obtener retroalimentación y planificar las acciones, etapas y objetivos del siguiente Sprint. Estas reuniones han sido esenciales para asegurar que el proyecto se mantuviera alineado con las expectativas, objetivos y necesidades del cliente.

Para llevar a cabo esta metodología, se asignaron unos roles:

- **Tutor (Cliente y Gestor de Proyecto)**: El tutor asumió un doble rol: cliente y gestor de proyecto. Como **cliente**, proporcionó los requisitos y feedback necesarios para asegurar que el desarrollo del proyecto seguía la línea impuesta, es decir, cumpliendo con las expectativas, necesidades y objetivos. Como **gestor de proyecto**, se aseguró que el equipo (el desarrollador), se mantuviera enfocado, manteniendo objetivos clave a realizar en cada Sprint.

- **Desarrollador del proyecto**: Yo asumí el rol de **desarrollador**. Mi responsabilidad fue implementar las tareas planificadas para cada Sprint, garantizar la calidad del código, realizar pruebas y asegurar que los entregables cumplieran con los requisitos definidos.

Para la **planificación y gestión del trabajo**, se ha utilizado **Trello**. Esta herramienta ha permitido tener una visión clara y organizada de cada Sprint, manteniendo los objetivos definidos con fechas límite de desarrollo. Gracias a Trello, se pudo seguir el progreso del proyecto, gestionar las tareas y asegurar el cumplimiento de los plazos establecidos, todo ello de manera eficiente y ordenada.

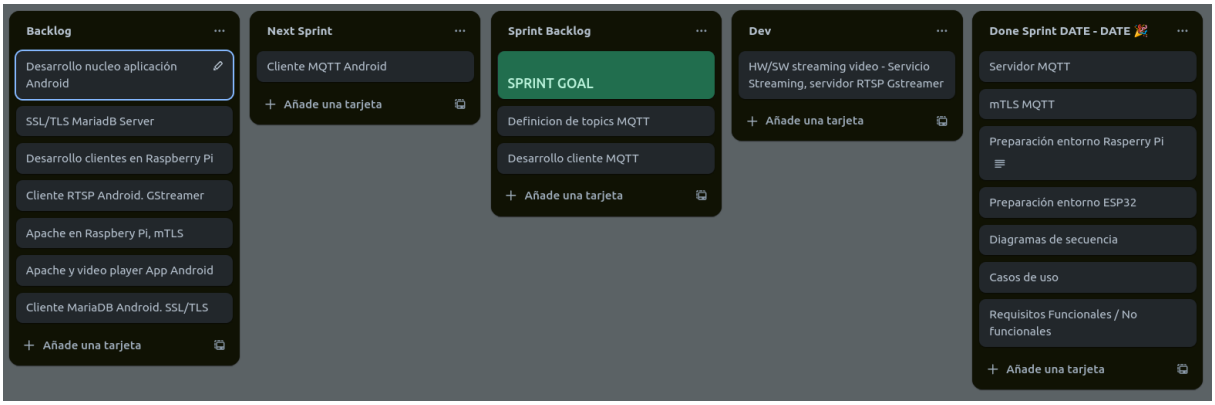


Figura 1: Ejemplo de Tareas del Proyecto en Trello.

GitHub ha sido la plataforma de **control de versiones** utilizada. GitHub ha facilitado el seguimiento de cambios en el código y el manejo de versiones del software desarrollado.

Gracias a GitHub, se asegura que siempre se trabaje sobre el código más actualizado, evitando conflictos y manteniendo un historial detallado de todas las modificaciones y mejoras realizadas en el proyecto. Esto ha permitido comparar archivos en caso de fallos, ver lo editado en cada commit anterior, y mantener una versión clara y organizada del proyecto en todo momento.

Aplicando esta metodología, se ha desarrollado el proyecto siguiendo un **proceso iterativo e incremental**, asegurando gran calidad en el resultado final, y una **continua alineación con las expectativas del cliente**.

4.2. Análisis y diseño de requisitos

El análisis de requisitos es una fase crítica del proyecto, ya que es donde se define qué debe hacer el sistema y cómo debe comportarse. Para tener bien organizados estos requisitos, se dividen en dos partes:

4.2.1. Requisitos funcionales

Un **requisito funcional** es una declaración de cómo debe comportarse un sistema. Define lo que el sistema debe hacer para satisfacer las necesidades o expectativas del usuario. Los requisitos funcionales se pueden considerar como características que el usuario detecta. Para este proyecto, tenemos:

- **Monitorización del entorno**: El ESP32 estará constantemente midiendo el entorno a través de sus sensores. En caso de detectar algo inusual si el sistema está activo (alarma activa), el ESP32 enviará inmediatamente información al servidor, alertando de un incidente.

- **Almacenamiento de datos y vídeo**: Si el sistema está activo y se detecta algún incidente por parte del ESP32, el servidor comenzará la grabación de vídeo, almacenará los datos relevantes del incidente, y el vídeo asociado a él.

- **Servicio Streaming**: El servidor iniciará el streaming si un cliente (App Android), se conecta y lo solicita.

- **Visualización en Tiempo Real**: Se podrá ver en la App Android el streaming de vídeo en tiempo real a través de la cámara que incorpora el servidor (Raspberry Pi).

- **Registro y Autenticación**: Los usuarios podrán registrarse en la App Android. Debe haber un sistema de autenticación para garantizar la seguridad y control de acceso al sistema.

- **Control del sistema**: Se podrá activar o desactivar el sistema de vigilancia desde la App Android, además de poder configurar qué sensores queremos que estén monitorizando el entorno.

- **Notificación de incidentes**: Se recibirán notificaciones en la App Android en caso de el sistema detecte alguna incidencia, alertando al usuario y mostrándole información en la notificación.
- **Registro de eventos**: Se podrá acceder desde la App Android a la información almacenada en la base de datos para cada incidente registrado. Esta información contiene datos detallados del incidente, como la hora, sensor que lo identificó y su valor, además de su grabación asociada.
- **Eliminación de eventos**: El usuario podrá eliminar eventos del registro de eventos.

4.2.2. Requisitos no funcionales

Los requisitos no funcionales describen las propiedades y cualidades del sistema, en lugar de las funciones específicas que debe realizar. Estos requisitos se centran en cómo debe comportarse el sistema y en los estándares que debe cumplir.

- **Cifrado de conexiones**: Se garantizará el cifrado de las conexiones y datos usando SSL/TLS en todas ellas.
- **Autenticación mTLS**: Todas las conexiones se configurarán con mTLS, para asegurar que tanto el cliente como el servidor se autentiquen mutuamente antes de establecer la conexión.
- **Autenticación de cada Servicio**: Cada servicio del sistema (MQTT, Apache, MariaDB, RTSP) se configurará para que, además de utilizar mTLS en las conexiones, también se necesite un usuario y contraseña válido para cada servicio, asegurando que sólo usuarios y dispositivos autorizados puedan interactuar con los componentes del sistema.
- **Codificación de vídeo**: La codificación de vídeo se realizará en formato H.264 para asegurar una transmisión eficiente y de alta calidad de las imágenes capturadas por la cámara.
- **Eficiencia streaming**: Se optimizará el pipeline de streaming para garantizar una transmisión de vídeo fluida y de baja latencia desde la Raspberry Pi a la aplicación Android, proporcionando una experiencia en tiempo real.

- **Acceso paralelo de vídeo**: La captura de vídeo y el streaming se diseñarán para que puedan funcionar al mismo tiempo, permitiendo ver el streaming, mientras se graba vídeo.
- **Cifrado de la base de datos**: Todos los datos almacenados en la base de datos estarán cifrados, protegiendo la información sensible y asegurando la privacidad.
- **Disponibilidad de servicios y sistema**: Se implementarán servicios de watchdogs y el uso de systemd para asegurar el reinicio automático del servicio en caso de caída, manteniendo el sistema siempre disponible.
- **Compatibilidad del hardware**: Se garantizará que todos los componentes de hardware, incluidos los sensores, cámara, Raspberry Pi y ESP32, funcionen de manera integrada y sin problemas.
- **Eficiencia**: El sistema será diseñado para ser eficiente en el uso de recursos, optimizando el consumo de energía para maximizar la duración y el rendimiento de los dispositivos.
- **Facilidad de uso**: La interfaz de la App Android será intuitiva y fácil de usar, proporcionando una experiencia de usuario amigable con instrucciones claras para la configuración y operación del sistema.

5

Diseño funcional del sistema

El diseño funcional del sistema se centra en definir cómo interactúan las diferentes partes y componentes, y cómo se deben llevar a cabo las funciones y requisitos descritos anteriormente.

Esta parte también es una parte crítica en el desarrollo del proyecto, ya que proporciona una representación detallada sobre cómo se deben implementar las diferentes partes del sistema, para cumplir con las expectativas y objetivos establecidos.

5.1. Diagrama General o de Servicios

En este diagrama se representa cómo **interactúan los servicios del sistema** con los diferentes dispositivos. Servirá para tener una visión general sobre el sistema y sus servicios. El diagrama se compone de diferentes capas o partes:

- **Presentation Layer (Verde)**: Esta capa representa lo que ve el usuario final. Incluye la App Android, que actúa como cliente de MQTT, RTSP GStreamer, MariaDB, y Apache.
- **Bussines Layer (Roja)**: En esta capa se realiza la lógica del sistema. Se incluyen los componentes principales que permiten las conexiones con los demás dispositivos, además de la lógica de las partes del sistema: App Android, ESP32 y sensores, y el cliente dentro del servidor. También se representan los diferentes clientes que tiene el servidor.

  La comunicación de los diferentes servicios entre cliente y servidor se realizan con propias APIs.
- **Data Layer (Azul)**: Esta capa es la responsable del almacenamiento y gestión de los datos del sistema. Incluye a la base datos y la API que interactúa con ella, además de la ruta donde Apache accede a las grabaciones de vídeo.

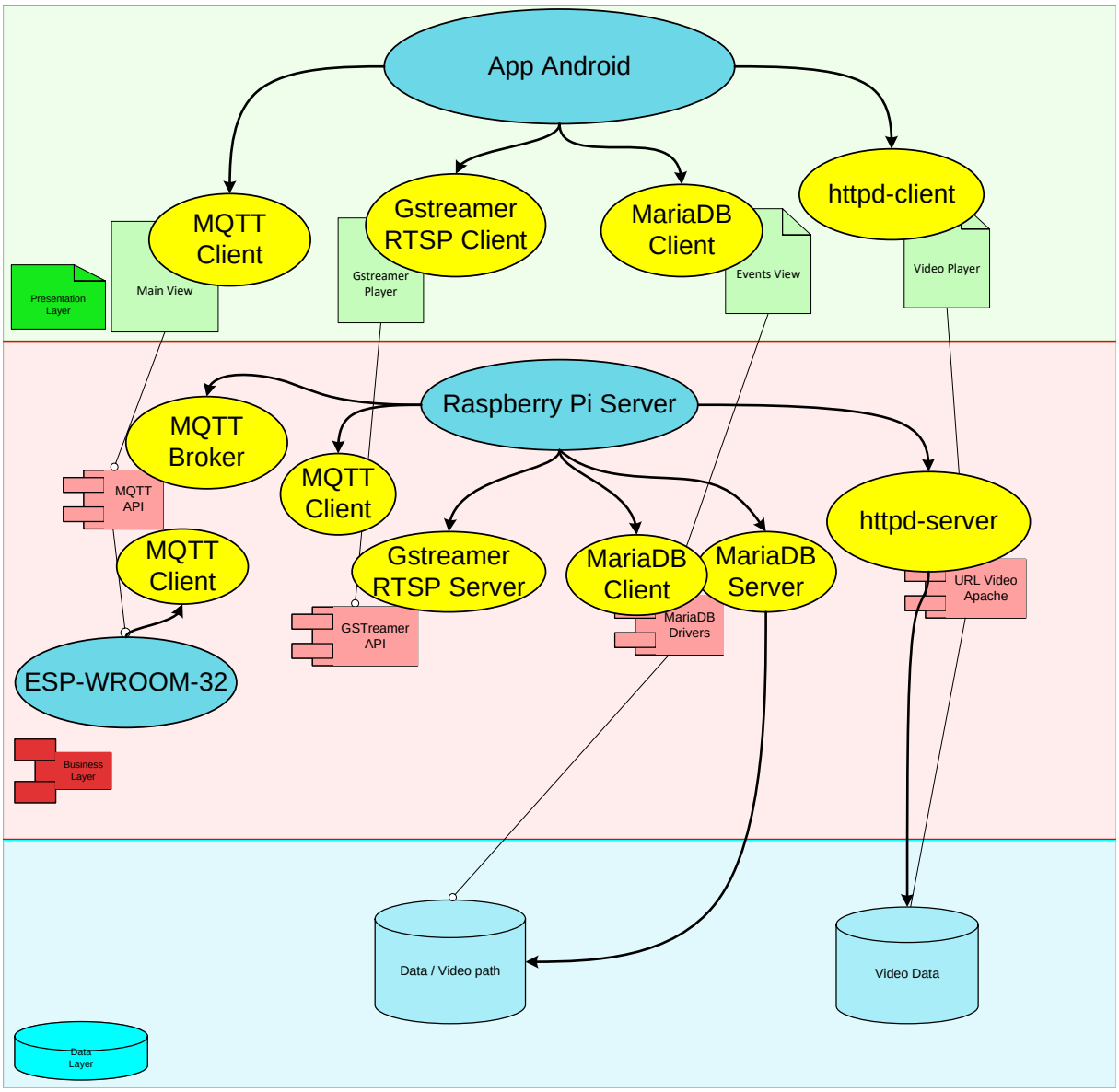


Figura 2: Esquema General de Servicios

Además de las capas, tenemos los siguientes elementos:

- **Dispositivos o clientes**: Se representan de color azul. Son la App Android, Raspberry Pi Server, y el ESP-WROOM-32.
- **Servicios**: Representados de color amarillo. Son MQTT, RTSP GStreamer, MariaDB y Apache, tanto cliente como servidor.
- **APIs**: Representadas de color rojo, una API (Application Programming Interface) es un intermediario entre dos sistemas, que permite que una aplicación se comunique con

otra y pida datos o acciones específicas. Tenemos: MQTT, RTSP GStreamer, MariaDB y Apache.

- **Datos**: Representados de color azul, tenemos los datos. Equivalen a la información que se almacena en MariaDB, y la ruta de almacenamiento de las grabaciones de vídeo.

5.2. Diagrama de Secuencia

En estos diagramas se mostrarán la **secuencias de pasos y mensajes** que interactúan entre los distintos componentes del sistema, tanto dentro de los dispositivos, como entre ellos.

Para tener una imagen general de la secuencia de pasos y mensajes en todo el sistema, dividiremos el diagrama en tres partes: inicio de sesión e interacción de cuentas con la base de datos, funcionalidades dentro de la App Android, y detección de una anomalía.

5.2.1. Inicio de Sesión, Manejo de Cuentas

Este diagrama de secuencia muestra como sería el proceso de **inicio de sesión** en la App Android, y las interacciones que ocurren entre los dispositivos y servicios.

Cuando pulsamos login con los datos introducidos, estos son enviados al servidor, dónde MariaDB los verificará. Si los datos son correctos, pasaremos de la vista de login a la vista general de la App Android, donde podremos manejar todo el sistema. En caso contrario, no nos permitirá acceder a la App, pudiendo volver a hacer el login de nuevo.

Cabe mencionar que la implementación del inicio de sesión mediante MariaDB es una **mejora planificada para trabajos futuros**. En la versión actual, el login se realiza **localmente** para propósitos de demostración y pruebas.

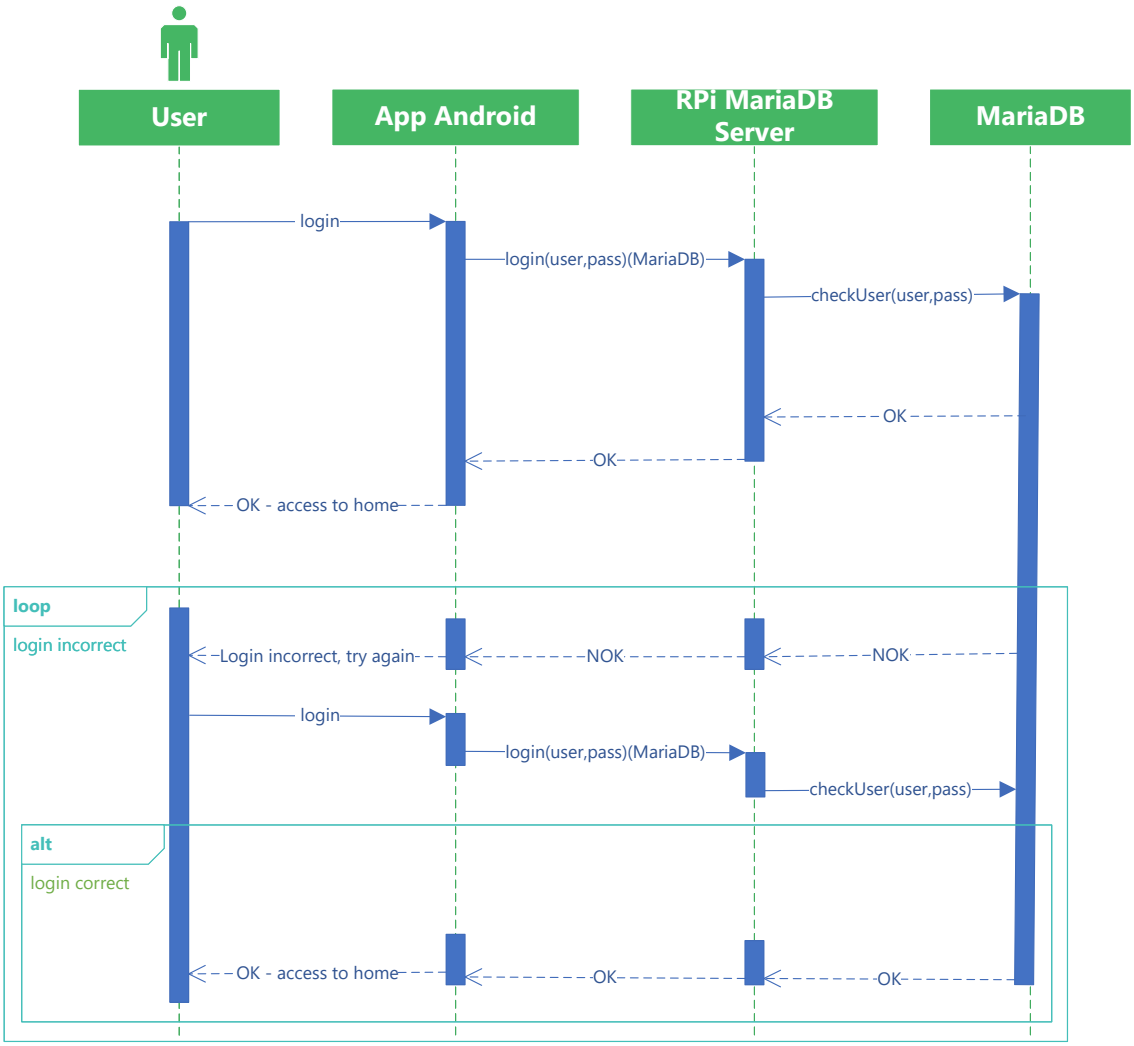


Figura 3: Diagrama de Secuencia Usuarios

5.2.2. Funcionalidades de la App Android

Este diagrama es bastante extenso, ya que incluye todas las funcionalidades que tiene el sistema con el usuario desde la App Android.

Se puede ver como es el intercambio de mensajes entre los dispositivos para las diferentes funcionalidades streaming, configuración de alarma, configuración de sensores, poder ver y eliminar ítems del registro de eventos, y cerrar sesión. Este diagrama asegura tener una visión general del funcionamiento del sistema con los dispositivos empleados:

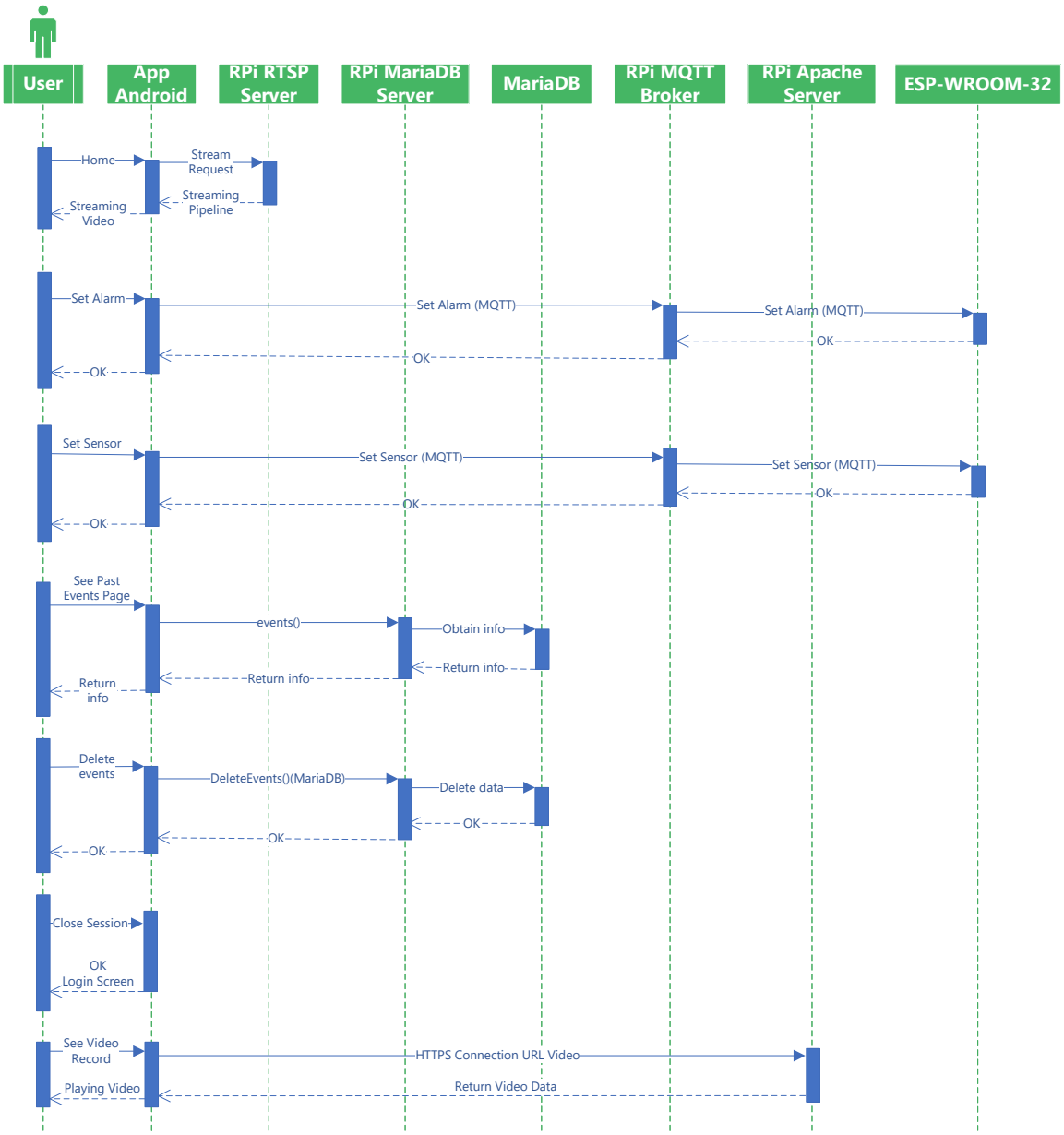


Figura 4: Diagrama de Secuencia Funcionalidades

5.2.3. Detección de una anomalía

En este diagrama se ilustra cómo funcionaría el sistema en caso de detectarse una anomalía.

El ESP32 está continuamente monitorizando el entorno con sus sensores en bucle. En el caso de detectarse una anomalía, el ESP32 inmediatamente envía un mensaje de alarma al servidor a través de MQTT. El cliente MQTT del servidor, suscrito al mismo topic de alarma

que la App Android, recibe este mensaje y realiza varias acciones: insertar la información en la base de datos y comenzar grabar vídeo. El cliente de la App Android al estar suscrito también, notificará al usuario con un mensaje de alerta, indicando que se ha detectado una anomalía en el entorno.

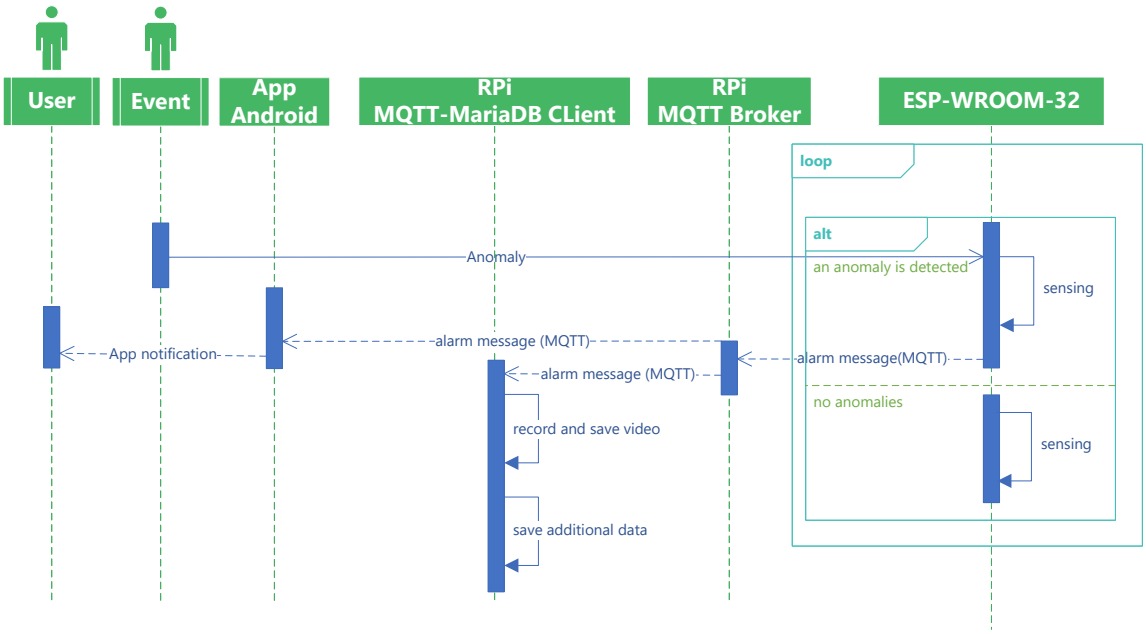


Figura 5: Diagrama de Secuencia Anomalía

5.3. Diagrama de Casos de Uso

Este diagrama representará las interacciones que puede hacer un usuario (actor) con el sistema, tanto el actor de la App Android, como el actor que hace disparar la alarma del sistema.

En este diagrama se utilizan dos relaciones entre casos de uso:

- **Include** (Incluir): Esta relación indica que un caso de uso siempre incluye la funcionalidad de otro. Es decir, el caso de uso base no puede realizarse sin incluir el comportamiento del caso de uso incluido. Por ejemplo, cuando se activa el "Sensing" al detectar una anomalía, siempre se incluye "Send Sensor Information"para el envío de la información al servidor del sensor, y "Alarm Trigger"para el disparo de alarma.

- **Extend** (Extender): Esta relación indica que un caso de uso puede ser extendido opcionalmente por otro caso de uso. Es decir, el caso de uso base funciona de manera independiente, pero en algunas ocasiones, se le añade el comportamiento adicional del caso de uso extendido. Por ejemplo, "Watch Streaming"puede extenderse con "Streaming", el streaming está siempre disponible, y cuando se conecta el usuario con "Watch Streaming", el servidor comienza a enviar el vídeo.

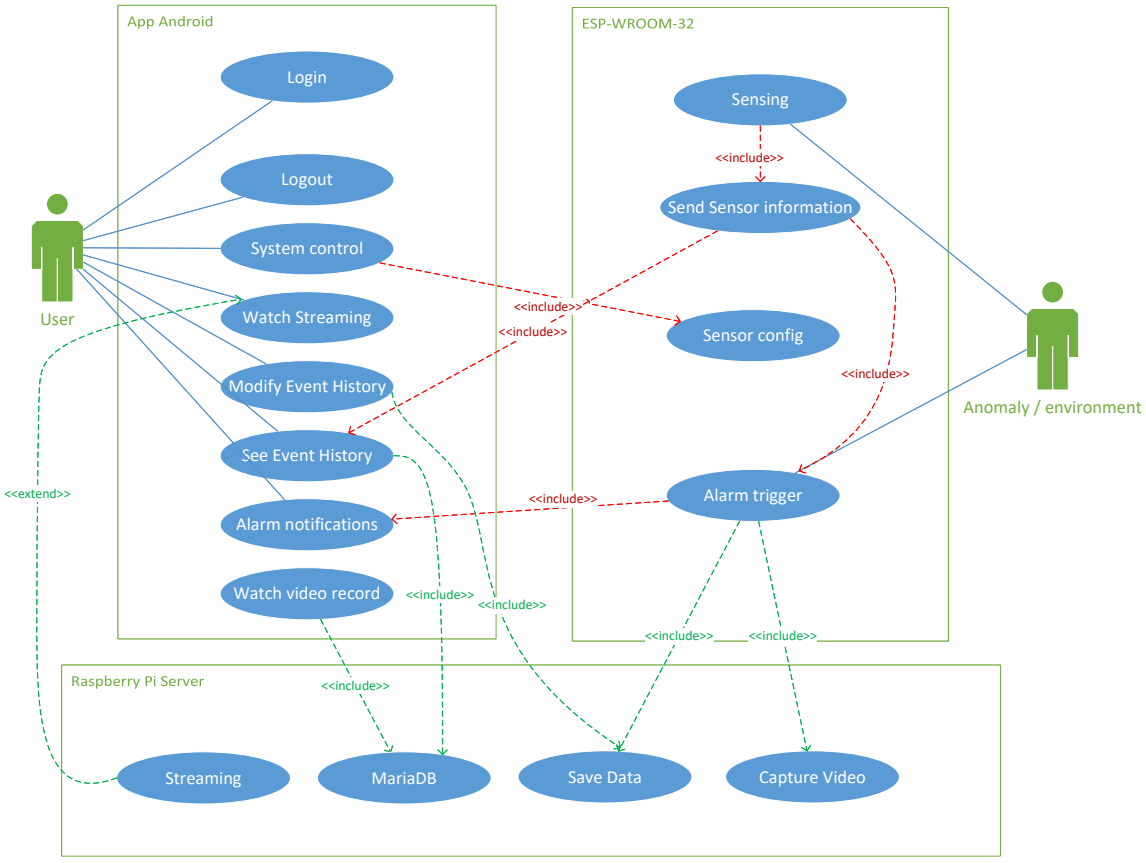


Figura 6: Diagrama de Casos de Uso

6

Diseño físico del sistema

El diseño físico del sistema se centra en la conexión e interacción de los componentes hardware que forman el proyecto. Este capítulo describe como los diferentes sensores, dispositivos y cámara se integran para que el sistema funcione correctamente.

Como ya se ha mencionado, se utilizan diferentes dispositivos:

- **ESP-WROOM-32**: Microcontrolador que obtiene datos de los sensores, los procesa y los envía al servidor.
- **Sensores**: Son un sensor **PIR HC-SR501** para detectar movimiento, un **interruptor magnético** que detecta la apertura de puertas y ventanas, y un **sensor de gas MQ-2** para detectar niveles peligrosos de gas o humo.
- **Raspberry Pi 3B+ y cámara**: Es núcleo del sistema. Actúa como servidor de diferentes servicios: MQTT, Apache, MariaDB, RTSP GStreamer. También actúa como cliente MQTT para procesar la información de incidentes del ESP32 y cliente de MariaDB para guardar la información recibida de ese incidente.

  La cámara conectada captura vídeo en alta definición, incluso en condiciones donde no hay luz visible, gracias a sus dos LEDs infrarrojos. Estos LEDs funcionan en conjunto con el filtro IR-cut de la cámara, que se encarga de bloquear la luz infrarroja durante el día para asegurar colores precisos, y se retira automáticamente en condiciones de poca luz para permitir la visión nocturna. Este vídeo es procesado por la Raspberry Pi a través de GStreamer, que lo prepara para ser guardado como grabación o transmitido mediante RTSP.
- **Smartphone Android**: Dispositivo sobre el que correrá la App desarrollada.

6.1. Conexionado ESP-WROOM-32

Como ya se mencionó en **Tecnologías y Herramientas Utilizadas** (ver capítulo 3), el ESP-WROOM-32 es un potente módulo basado en el chip ESP32 de Espressif, y es la solución perfecta para este proyecto.

Este módulo gestionará toda la lógica de los sensores, y se conectará al servidor mediante MQTT para enviar la información recibida de estos.

El ESP-WROOM-32 cuenta con una amplia gama de pines GPIO en los que se conectarán los diferentes sensores. Además de los pines VCC (5 voltios), GND (Negativo, tierra) y 3.3V (3.3 voltios), se necesitarán tres pines para la lectura de los sensores, el GPIO 4, 13 y 36.. Estos tres pines se configurarán en el capítulo **Programación del ESP32 y Sensores** (ver capítulo 9). Hay que tener en cuenta que la salida del Sensor de Gas MQ-2 es analógica, por lo que se deberá conectar a un pin que soporte **ADC (Analog to Digital Converter)**, para convertir la señal analógica en digital. El ESP32, se alimentará con un cable USB-C, a **5 voltios**.

En las siguientes figuras, se representa como es el Pinout del ESP-WROOM-32 y su conexionado con los sensores:

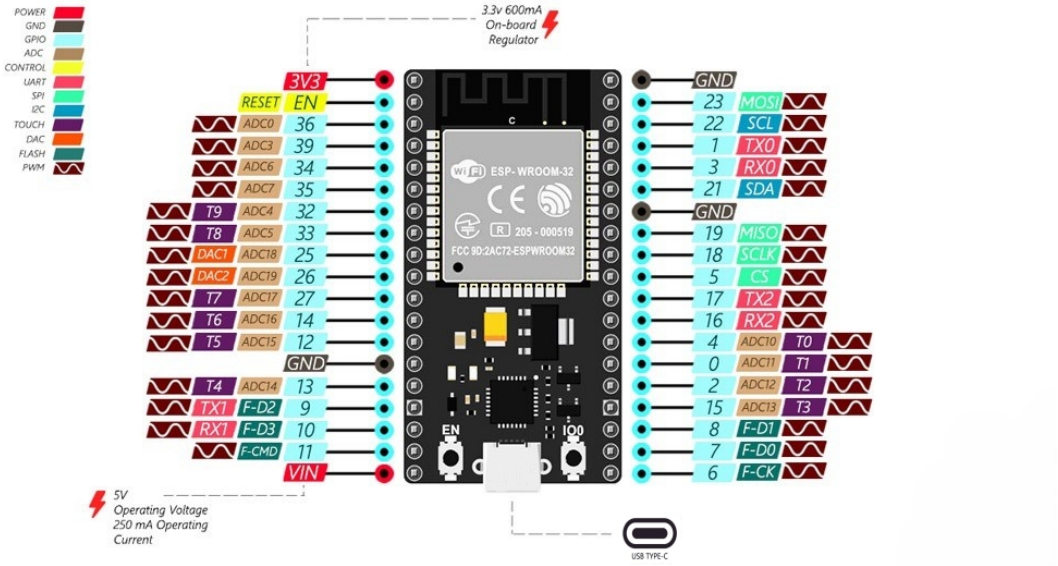


Figura 7: Pinout ESP-WROOM-32

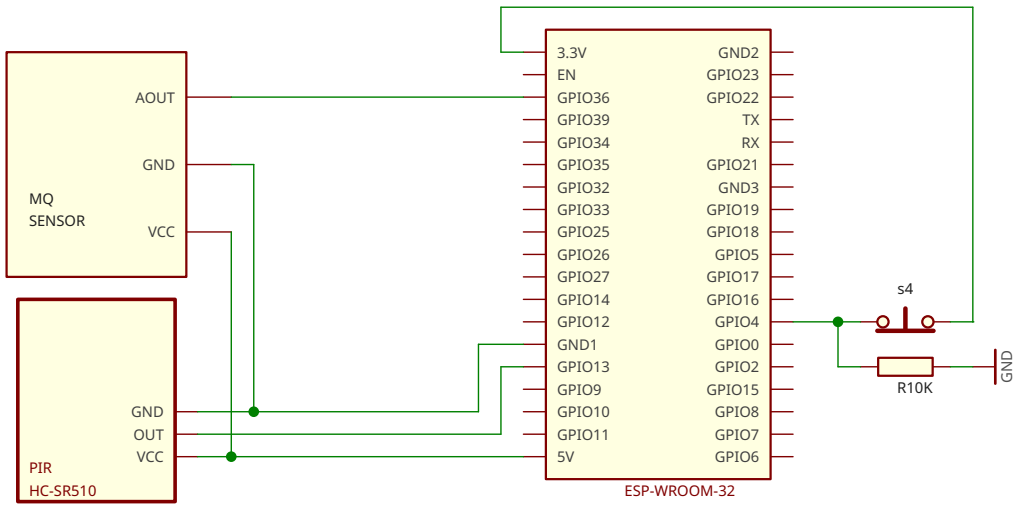


Figura 8: Conexionado ESP-WROOM-32

- **Sensor de gas MQ-2**: Este sensor irá alimentado a **5 voltios**, y su salida analógica irá **conectada al GPIO36**. Aunque este sensor incorpore un potenciómetro, no será utilizado. Este potenciómetro sirve para ajustar la sensibilidad del sensor si se usa su pin digital. Se tendrá en cuenta especialmente esta conexión a la hora de configurar el sensor en el código del ESP32, ya que hará falta configurar un canal **ADC (Analog to Digital Converter)**.

- **Senror PIR HC-SR501**: Se alimentará a **5 voltios**, y su salida digital irá conectada al **GPIO13** del ESP32. Como se puede apreciar en la Figura 10, podremos ajustar el sensor de diferentes maneras:

  - **Sensitivity Adjust (Sensibilidad)**: Permite controlar la distancia a la que el sensor puede detectar movimiento. Se configurará al **máximo**, girando el potenciómetro en el sentido de las agujas del reloj **(7 metros aproximadamente)**.

- **Time-Delay Adjust (Retardo de tiempo)**: Controla el tiempo que la salida del sensor permanece en estado alto (HIGH) después de haber detectado movimiento. Si giramos en sentido de las agujas del reloj, incrementa. Al contrario, disminuye. Se ajustará el potenciómetro para obtener un retardo de unos **5 segundos aproximadamente**.
- **Jump Trigger (Modo de Disparo)**: Determina cómo el sensor responde al detectar movimiento: **L (Low, Disparo Único)**: El sensor enviará una señal alta (HIGH) solo una vez cuando detecte movimiento y permanecerá en estado bajo (LOW) una vez que el temporizador termine. **H (High, Disparo Repetido)**: el sensor enviará una señal alta (HIGH) mientras continúe detectando movimiento, y el temporizador se reiniciará cada vez que se detecte movimiento adicional. La configuración del Jump Trigger será **H, Disparo Repetido**.

- **Interruptor magnético**: Estará alimentado a **3.3 voltios**, y se conectará al **GPIO4**. Este sensor tendrá configurada una **resistencia de 10k ohmios**, actuando como una **resistencia pull-down**, asegurando que el estado del pin (GPIO4) siempre sea bajo cuando el interruptor esté abierto.

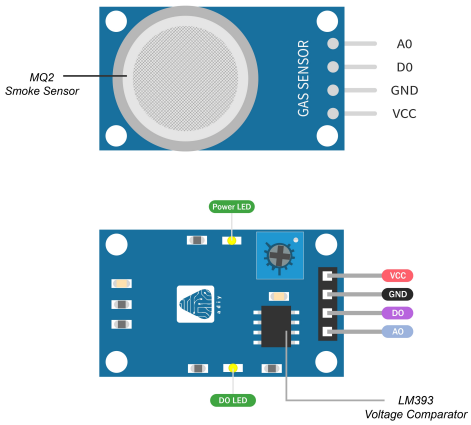


Figura 9: Pinout Sensor de Gas MQ2

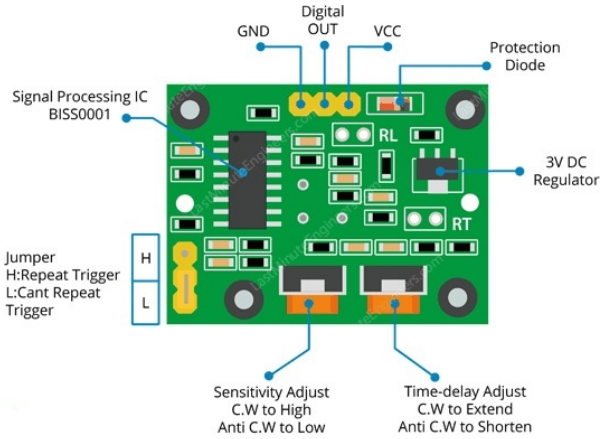


Figura 10: Pinout Sensor de Presencia PIR HC-SR501



Figura 11: Interruptor Magnético

6.2. Conexionado Raspberry Pi 3B+

La Raspberry Pi se conectará a Ethernet para asegurar una conexión de red estable y rápida. Esto es importante para mantener fluidez en las comunicaciones del sistema con los demás dispositivos y permitir el acceso remoto. Aunque la Raspberry Pi tiene numerosos puertos, sólo utilizaremos el **módulo de cámara CSI** para la cámara IR-Cut, **el lector de la tarjeta Micro SD** para la tarjeta Micro SD y Ethernet para la conexión a internet.

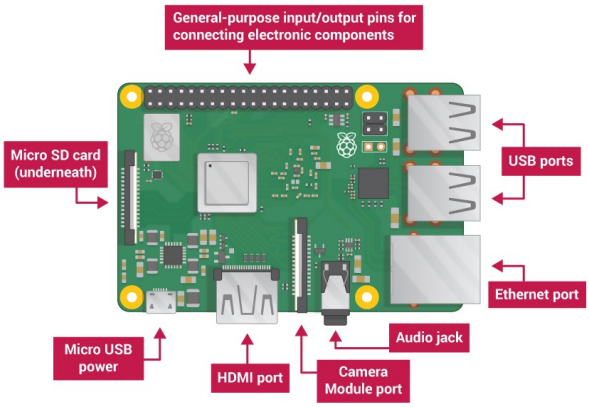


Figura 12: Puertos Raspberry Pi 3B+

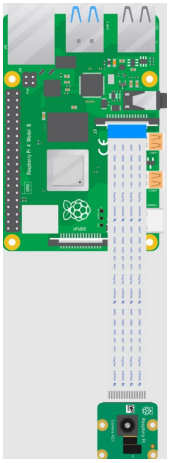


Figura 13: Conexionado Raspberry Pi 3B+



Figura 14: Cámara IR-Cut 1080p y LEDs IR

6.3. Sistema final

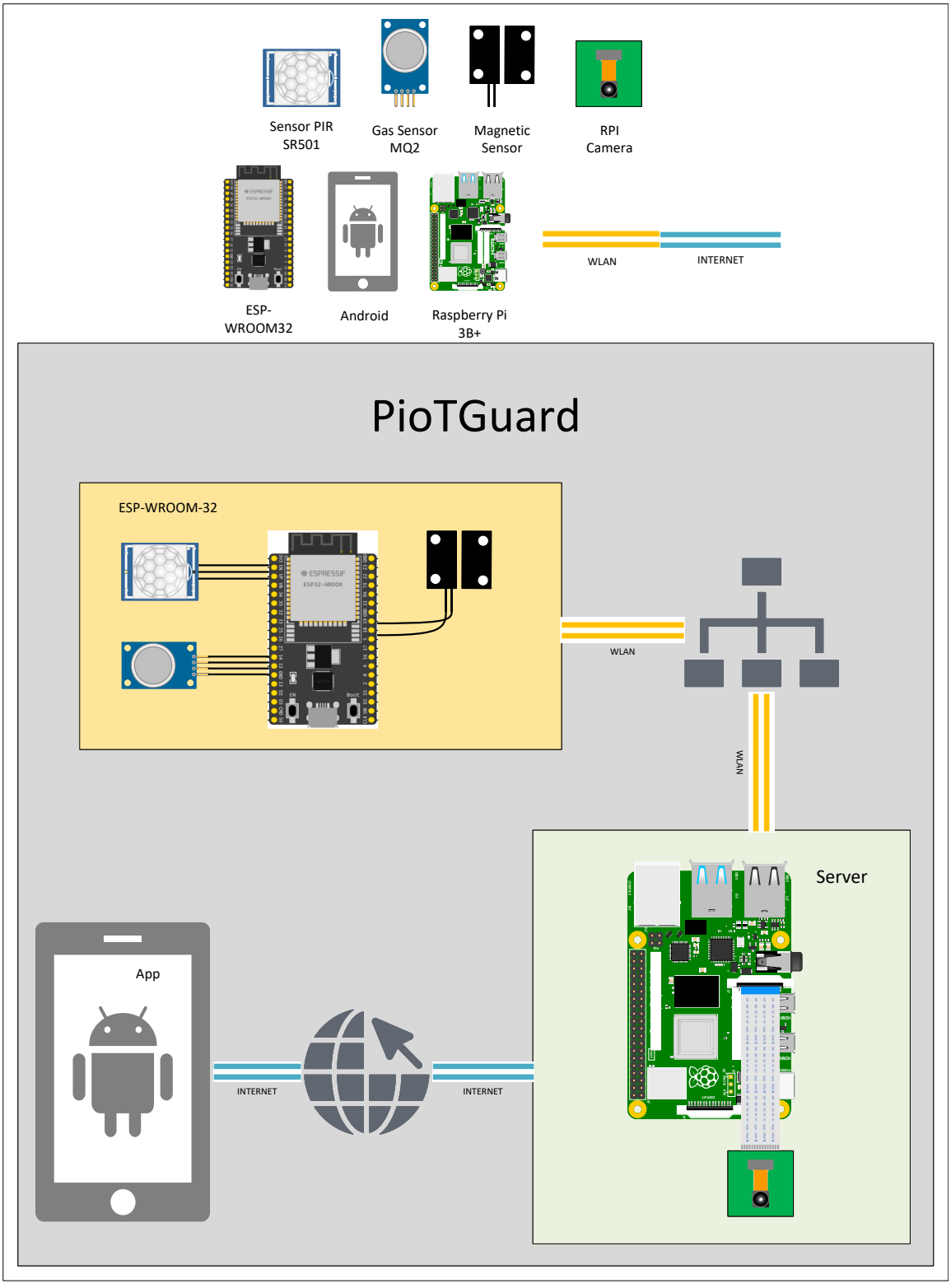


Figura 15: Esquema Físico General

7

Certificados SSL/TLS y autenticación mTLS

Como ya hemos explicado anteriormente, para garantizar la seguridad del sistema, es crucial proteger todas las comunicaciones entre los dispositivos utilizando certificados SSL/TLS y autenticación mTLS. Este capítulo describe cómo ha sido el proceso de generación y configuración de los certificados necesarios para implementar en el sistema el uso de SSL/TLS y mTLS, mejorando la seguridad de las conexiones y, por tanto, la del sistema completo.

Aunque el sistema implementa estos protocolos funcionando correctamente, la configuración actual de los certificados y del sistema no es la más segura que se puede tener a día de hoy. Debido a las limitaciones de tiempo del TFG, no se han podido completar todas las mejoras necesarias para asegurar el nivel más alto de seguridad disponible actualmente. Por este motivo, no sería ideal llevar este sistema a un entorno de producción, ya que un sistema de seguridad o vigilancia es un **objetivo potencial de ataques**. Sin embargo, para un entorno de desarrollo y con la configuración actual, este sistema proporciona un nivel de seguridad más que suficiente.

Para que el sistema sea realmente seguro y apto para un entorno de producción, sería necesario realizar varias mejoras. Primero, se deben actualizar los algoritmos de cifrado, reemplazando RSA por Ed25519 y DES3 por AES, que son algoritmos más seguros. Actualmente, AES es el estándar para el cifrado. Además, es importante incluir extensiones en los certificados de la CA que indiquen que se trata realmente de una CA y que el certificado se utiliza para firmar otros certificados (**KeyUsage**), reforzando así la autenticidad y validez de los certificados en el sistema.

Implementar mejoras como la actualización de los algoritmos de cifrado, utilizar Keyring en el servidor y mejorar manejo de credenciales, son esenciales para llevar este sistema a un entorno real. Estas y otras mejoras están descritas en el **Capítulo** 12.

Un **certificado SSL/TLS** es un objeto digital que permite a los sistemas verificar la identidad de otros sistemas y establecer conexiones de red cifradas utilizando los protocolos **Secure Sockets Layer (SSL)** y **Transport Layer Security (TLS)**. Estos certificados se generan utilizando un sistema criptográfico conocido como **infraestructura de clave pública (PKI)**.

La **PKI** permite que una **entidad confirme la identidad de otra** mediante el uso de estos certificados, siempre que ambas confíen en un tercero, la **autoridad de certificación (CA)**. Los certificados SSL/TLS actúan como **credenciales digitales** que aseguran las comunicaciones de red y validan la identidad de sitios web en Internet, así como de recursos en redes privadas.

Los **protocolos SSL/TLS** son **protocolos de comunicación** que permiten a los sistemas comunicarse de manera segura a través de Internet. Los certificados SSL/TLS permiten que un sistema verifique la identidad del otro (opcionalmente que se verifiquen ambos, mTLS), estableciendo conexiones cifradas entre ellos. Al utilizar estos protocolos, se garantiza que los datos transmitidos sean seguros y estén protegidos.

7.1. Principios de SSL/TLS

Cifrado

El cifrado consiste en transformar un mensaje original de manera que solo pueda ser leído por el destinatario autorizado. Por ejemplo, se puede cambiar la palabra “sensor” a “ugpugq” desplazando cada letra dos posiciones hacia adelante en el alfabeto. El destinatario, que conoce la regla (o clave), puede revertir el desplazamiento de cada letra para leer la palabra original. El cifrado SSL/TLS se basa en este principio utilizando criptografía de clave pública, que emplea dos claves diferentes para cifrar y descifrar un mensaje.

Criptografía de clave pública

La criptografía de clave pública es un método en el que se utilizan dos claves distintas: una **clave pública para cifrar** el mensaje y una **clave privada para descifrarlo**. Solo el desti-

natario, que posee la clave privada correspondiente, puede descifrar el mensaje. Esta técnica asegura que la comunicación sea segura y que solo el destinatario especificado pueda leer el contenido del mensaje.

Infraestructura de clave pública

La infraestructura de clave pública (PKI) permite a una entidad establecer la identidad de otra mediante el uso de certificados digitales, siempre que ambas partes confíen en un tercero conocido como **autoridad certificadora (CA)**. La CA verifica la validez del certificado y autentica a ambas partes antes de que comience la comunicación. Esto se realiza emitiendo certificados que contienen las claves públicas de las partes involucradas y firmándolos digitalmente para garantizar su autenticidad.

7.2. Mutual TLS

mTLS (Mutual TLS) es una extensión del protocolo TLS que añade una capa adicional de seguridad mediante la autenticación mutua entre los dispositivos que se comunican. En una configuración estándar de TLS, sólo el servidor es autenticado por el cliente mediante un certificado digital. Sin embargo, en mTLS, tanto el cliente como el servidor y se autentican mutuamente.

7.3. Funcionamiento de SSL/TLS y mTLS

En una comunicación segura mediante SSL/TLS, cuando dos sistemas desean comunicarse de manera segura, primero intercambian sus certificados digitales. Cada entidad (cliente/servidor) verifica el certificado de la otra parte utilizando el certificado de la CA que ambos poseen y en la cual confían. Una vez que ambas partes se han autenticado, se establece una conexión cifrada usando sus claves públicas y privadas. Además, es crucial que cada parte confirme que el nombre de la otra entidad es el correcto, con la cual desean comunicarse. Esta verificación se realiza fuera del protocolo SSL/TLS, después de la autenticación mutua. De esta forma, la conexión asegura que cualquier dato transmitido entre los sistemas esté protegido.

1. **Establecimiento de la conexión**:

    - Antes de iniciar la transmisión de datos, SSL/TLS realiza un proceso conocido como "**handshake**", mediante el cual se autentican una, o ambas entidades que van a establecer una comunicación segura.

2. **Cifrado de datos**:

    - Una vez que la conexión se ha establecido, todos los datos enviados entre el cliente y el servidor se cifran utilizando las claves de sesión acordadas. Esto protege los datos de ser interceptados o alterados por terceros.

      El cifrado garantiza que cualquier dato interceptado mientras se transmite no pueda ser leído, ya que solo las partes con las claves correctas pueden descifrar la información.

3. **Integridad de datos**:

    - SSL/TLS también garantiza que los datos enviados no sean modificados durante la transmisión. Utiliza técnicas como los **códigos de autenticación de mensajes (MAC)** para asegurar que los datos recibidos sean exactamente los mismos que los enviados, sin modificaciones.

4. **Finalización de la Conexión:**

    - Al finalizar la comunicación, SSL/TLS garantiza que la conexión se cierre de manera segura, impidiendo que las claves de sesión puedan ser reutilizadas para futuros ataques.

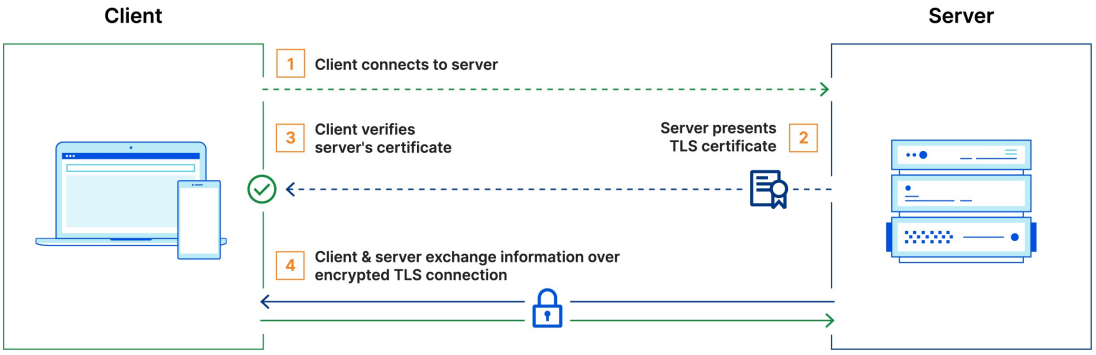


Figura 16: Funcionamiento de una conexión SSL/TLS

7.4. Utilidad y Necesidad de mTLS

El uso de **mTLS (Mutual Transport Layer Security)** en este proyecto es fundamental para garantizar la seguridad y la integridad de la información transmitida entre los diferentes dispositivos y servicios:

La **autenticación del servidor** es fundamental para asegurar que el cliente reciba y envíe sus datos, como la detección de intrusos, gas o humo, al servidor adecuado. Esto evita que un servidor fraudulento se haga pasar por el verdadero para interceptar la información. Con mTLS, el cliente puede verificar que está comunicándose con el servidor correcto, eliminando el riesgo de que los datos sean capturados por un servidor falso.

Por otro lado, **la autenticación del cliente** garantiza al servidor que la información recibida o enviada proviene del dispositivo adecuado. Esto es esencial para evitar que un dispositivo no autorizado reciba datos sensibles, y envíe otros que puedan comprometer al sistema. Al validar la identidad del cliente, el servidor puede estar seguro de que las alertas y los estados del sistema son auténticos y no han sido falsificados.

El **cifrado** de la comunicación, combinado con la autenticación mutua, asegura que la información transmitida entre el cliente y el servidor es confidencial y solo es accesible para las entidades adecuadas. Esto es particularmente importante dado que la información que se maneja es extremadamente sensible. El cifrado evita que cualquier interceptación de los datos en tránsito pueda ser leída por actores no autorizados.

Finalmente, **la comprobación de integridad** garantiza que la información recibida no ha sido modificada durante su transmisión. Esto significa que cualquier intento de alterar los datos en tránsito será detectado, asegurando que la información que llega a su destino es exacta y no ha sido comprometida.

7.5. Contenido de los certificados

Los certificados creados para este proyecto se componen de varios elementos clave que aseguran su autenticidad y funcionalidad. A continuación, se describe el contenido y la configuración de los certificados creados para la CA, el servidor y los clientes, incluyendo los datos específicos introducidos en cada uno.

7.5.1. Contenido de los certificados de las CAs

1. **Clave Privada de la CA:**
    - Generada utilizando RSA con una longitud de 4096 bits.
    - Cifrada con el algoritmo DES3 protegida con contraseña.
2. **Certificado de la CA:**
    - Certificado autofirmado válido por 10 años (3650 días).
    - Información introducida:
        - País (C): ES
        - Estado o Provincia (ST): Malaga
        - Localidad (L): Malaga
        - Organización (O): PioTGuard
        - Unidad Organizativa (OU): CA
        - Nombre Común (CN): Nombre descriptivo de la CA para cada servicio
        - Dirección de Correo Electrónico: dirección de correo de contacto

7.5.2. Contenido de los certificados del servidor

1. **Clave Privada del Servidor:**
   - Generada utilizando RSA con una longitud de 4096 bits.
   - No cifrada en la parte del servidor.
2. **Certificado del Servidor:**
   - Certificado firmado por la CA.
   - Información introducida y configurada en el archivo **openssl.cnf** (ver Apéndice A.1).
   - El certificado incluye **extensiones SAN** (Subject Alternative Names) para permitir múltiples nombres de host y direcciones IP.

7.5.3. Certificados de los Clientes

1. **Clave Privada del Cliente:**
   - Generada utilizando RSA con una longitud de 4096 bits.
   - Cifrada con el algoritmo DES3 para la App Android y el cliente dentro del servidor, y AES-128 para el ESP32.
2. **Certificado del Cliente:**
   - Certificado firmado por la CA.
   - Información introducida al crear el CSR del cliente:
     - País (C): ES
     - Estado o Provincia (ST): Malaga
     - Localidad (L): Malaga
     - Organización (O): PioTGuard
     - Unidad Organizativa (OU): Cliente específico (por ejemplo, App)
     - Nombre Común (CN): Nombre único para cada cliente
     - Dirección de Correo Electrónico: dirección de correo de contacto

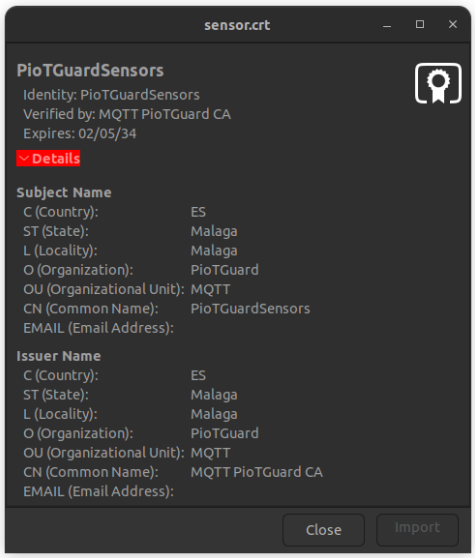


Figura 17: Ejemplo de un certificado del cliente

Con esta configuración, se garantiza que todos los certificados creados proporcionen la autenticidad y seguridad necesarias para las comunicaciones del sistema, generando cada certificado con información detallada que identifica de manera única a cada entidad (CA, servidor y clientes) y asegura la integridad de las conexiones. Resumiendo el proceso tenemos:

- **Certificados de la CA:** Utilizados para firmar los certificados del servidor y los clientes. Proporcionan una autoridad de confianza para verificar la autenticidad de los certificados.

- **Certificados del Servidor:** Configurados con el archivo **openssl.cnf** para incluir extensiones SAN (ver Apéndice A.1). Permiten que el servidor se autentique ante los clientes y establezca conexiones cifradas.

- **Certificados de los Clientes:** Firmados por la CA para asegurar su autenticidad. Permiten que los clientes se autentiquen ante el servidor y establezcan conexiones cifradas. Para los dispositivos ESP32, se ha utilizado AES para cifrar las claves debido a limitaciones.

7.6. Generación de certificados

Una vez claro el contenido de cada uno de los certificados, pasamos a su generación. Dependiendo de si es CA, cliente, o servidor, se necesitarán hacer unos pasos u otros. Para llevar a cabo esta generación, se ha utilizado la herramienta **OpenSSL**:

- **Certificados de la Autoridad de Certificación (CA)**

  1. **Clave Privada de la CA:** Se genera una clave privada RSA de 4096 bits, cifrada con DES3.
  2. **Certificado de la CA:** Se crea un certificado autofirmado válido por 10 años, incluyendo información relevante descrita anteriormente.

- **Certificados del Servidor**

  1. **Clave Privada del Servidor:** Se genera una clave privada RSA de 4096 bits.
  2. **Solicitud de Firma de Certificado (CSR):** Se crea una CSR con extensiones SAN especificadas en el archivo de configuración.
  3. **Firma del CSR con la CA:** Se firma el CSR con la CA de su servicio, generando el certificado **.crt** que se usará en la configuración del servicio.

- **Certificados de los Clientes**

  1. **Clave Privada del Cliente:** Se genera una clave privada RSA de 4096 bits, cifrada con DES3 para la mayoría de los clientes y con AES-128 para los dispositivos ESP32.
  2. **Solicitud de Firma de Certificado (CSR):** Se crea una CSR para cada cliente, incluyendo información descrita anteriormente.
  3. **Firma del CSR con la CA:** Se firma el CSR con la CA de su servicio, generando el certificado **.crt** que se usará en la configuración del servicio.

- **Certificados para la App Android**

  - Los certificados que se utilizarán en la App Android se convertirán a formato .p12 para facilitar su manejo e implementación en la App.

Teniendo los certificados generados y configurados, el siguiente paso es integrar estos certificados en los clientes y el servidor. Esto implica configurar cada dispositivo con los certificados correspondientes para utilizar SSL/TLS y mTLS. En particular, los certificados utilizados en la App Android deben ser convertidos al formato .p12.

De esta manera, se garantiza que tanto el servidor como los clientes puedan establecer conexiones seguras y verificar la identidad de las partes involucradas. La configuración de los certificados en cada dispositivo se describirá en las siguientes secciones.

8

Preparación del Servidor

Para que la Raspberry Pi actúe como servidor de los servicios necesarios en este sistema, primero se deberá instalar un sistema operativo (SO). En este caso, se utilizará **Raspberry Pi OS**, un SO diseñado específicamente para las Raspberry Pi. Raspberry Pi OS se puede descargar desde la web oficial de Raspberry **[1]**.

Una vez descargado, el SO se graba en una tarjeta Micro SD, que luego se inserta en la ranura correspondiente de la Raspberry Pi. Después de configurar y cargar el SO, se configuran los usuarios y se instalan los paquetes necesarios para el sistema.

8.1. UFW

Para poder acceder a la Raspberry Pi desde Internet añadiéndole una capa adicional de seguridad, se instalará y configurará **UFW (Uncomplicated Firewall)**, un firewall simple y fácil de usar para dispositivos Linux. Este firewall se configurará para abrir sólo los puertos necesarios de los servicios que se utilizarán, protegiendo el sistema. También será necesario abrir el **puerto 22 de SSH** para poder acceder remotamente a la Raspberry Pi.

Los puertos que se deberán abrir son:

- **SSH**: Puerto 22 para acceso remoto.
- **MQTT**: Puerto 8883 para el broker MQTT.
- **RTSP**: Puerto 8554 para el streaming de vídeo mediante RTSP.
- **MariaDB**: Puerto 3306 para la base de datos MariaDB.
- **HTTPS**: Puerto 443 para ver las grabaciones de vídeo.

8.2. Mosquitto

Mosquitto es un broker MQTT que facilita la comunicación entre dispositivos en un sistema IoT utilizando el protocolo MQTT (Message Queuing Telemetry Transport), donde su funcionamiento se basa en un modelo de **publicación y suscripción**.

Para implementar el **broker MQTT** es decir, el servidor de Mosquitto, será necesario instalar los paquetes correspondientes de Mosquitto. Los paquetes que se deben instalar son: **libmosquitto-dev**, **mosquitto-clients** y **mosquitto-dev**. Una vez instalados estos paquetes, solo quedará configurar Mosquitto.

La configuración de Mosquitto se encuentra en la ruta **/etc/mosquitto**. Dentro de esta ruta, hay varias carpetas importantes: **certs/**, donde se colocarán los certificados necesarios y el archivo **passwd** para el servidor de Mosquitto, y **conf.d/**, donde se creará el archivo de configuración de Mosquitto.

Se deberá configurar Mosquitto para que acepte conexiones desde Internet, especificar las rutas de los certificados para utilizar SSL/TLS, activar mTLS y especificar la ruta del archivo de autenticación. Para este último, primero es necesario crearlo y añadirle los usuarios y contraseñas necesarios, que en este caso son **app** (App Android), **localhost** (cliente dentro de la Raspberry Pi) y **sensor** (ESP32). El contenido del archivo de configuración de Mosquitto está detallado en el Apéndice A.2.

Con estos pasos y configuración, si todo es correcto, se tendría un servidor MQTT con mTLS implementado, y sólo podrán acceder los usuarios con su contraseña especificados en el archivo **passwd**, presentando los certificados correctos. Destacar que todos los clientes del sistema utilizarán el **máximo QoS**, para asegurar que los mensajes son recibidos correctamente.

8.3. GStreamer

Para poder configurar un servidor **RTSP (Real Time Streaming Protocol)** con GStreamer, primero será necesario instalar varios paquetes en el sistema. Estos incluyen: **gstreamer1.0-tools**, **gstreamer1.0-plugins-base**, **gstreamer1.0-plugins-good**, **gstreamer1.0-plugins-bad**, **gstreamer1.0-plugins-ugly**, **gstreamer1.0-libav**, **libgstreamer1.0-dev**, **libgstreamer-plugins-base1.0-dev**, además del paquete específico para RTSP: **gstreamer1.0-rtsp**.

Los primeros paquetes proporcionan la base de GStreamer: plugins para la codificación y decodificación de vídeo, y las bibliotecas necesarias para desarrollar programas con GStreamer. El último paquete contiene los archivos necesarios para poder desarrollar clientes y servidores RTSP, es decir, el streaming a través de RTSP.

Dado que la Raspberry Pi utiliza Raspberry Pi OS, un sistema operativo basado en UNIX, solo se podrá acceder al vídeo de la cámara a través de un proceso a la vez. En otras palabras, no se puede acceder al mismo dispositivo simultáneamente desde diferentes procesos. Esto significa que solo se podrá grabar vídeo o hacer streaming, pero no ambas cosas a la vez. El dispositivo de vídeo (la cámara) que nos proporciona el SO se encuentra en **/dev/video0**.

Uno de los requisitos del proyecto es la capacidad de hacer streaming de vídeo mientras se graba. Para solucionar esta limitación, se crearán dos dispositivos virtuales usando **v4l2loopback**, que recibirán la fuente de vídeo utilizando GStreamer. La idea es usar GStreamer para enviar el vídeo de **/dev/video0** a dos dispositivos virtuales: **/dev/video1** para el streaming y **/dev/video2** para la grabación. Para lograr esto, se necesitará:

- **Crear los dispositivos virtuales**: Instalaremos un módulo llamado **v4l2loopback** para crear los dispositivos virtuales. Este módulo permite la creación de dispositivos virtuales de vídeo, en nuestro caso, para el vídeo de la cámara.
- **Uso de GStreamer con Tee**: Utilizaremos GStreamer para dividir la fuente de vídeo de **/dev/video0** y enviarla a los dos dispositivos virtuales:

```
gst-launch-1.0 v4l2src device=/dev/video0 ! videoconvert ! videoscale
↪ ! video/x-raw,format=I420,width=640,height=480,framerate=30/1 ! tee
↪ name=t ! queue ! v4l2sink device=/dev/video1 sync=FALSE t. ! queue
↪ ! v4l2sink device=/dev/video2 sync=FALSE
```

Este comando toma el vídeo de **/dev/video0**, lo convierte y escala a un formato compatible para **v4l2loopback**, y luego usa **tee** para dividir la señal en dos colas (**queue**), enviando una a **/dev/video1**, que será utilizado como fuente de vídeo para el streaming, y la otra a **/dev/video2** como fuente de vídeo para la grabación.

Con esta configuración, se cumple con el requisito de realizar streaming de vídeo mientras se graba simultáneamente, gracias a la herramienta v4l2loopback y GStreamer para gestionar la fuente de vídeo.

Ya solo queda desarrollar el programa del servidor RTSP, y el de la grabación de vídeo, que serán descritos en el capítulo 10. En cuanto a la configuración del streaming con SSL/TLS, se hará de manera diferente en comparación con los demás servicios, se configurará directamente en el programa del servidor RTSP desarrollado.

8.4. MariaDB

8.4.1. Configuración

Para configurar MariaDB con soporte SSL/TLS y mTLS, será necesario instalar los paquetes correspondientes de MariaDB, en concreto **mariadb-server**.

La configuración de MariaDB se encuentra en la ruta **/etc/mysql/mariadb.conf.d/**. Dentro de esta ruta, hay varios archivos de configuración importantes,**50-server.cnf**, y **50-clients.cnf** donde se agregará la configuración SSL/TLS para los clientes y servidor, y mTLS para el servidor.

Se deberá configurar MariaDB para que acepte conexiones seguras desde Internet, especificar las rutas de los certificados para SSL/TLS, activar mTLS y definir la configuración de autenticación. Para esto, se añadirá al archivo **50-server.cnf** la información especificada (ver Apéndice A.3).

Además, para que la información sea cifrada con todos los clientes, se necesitará modificar el archivo **50-client.cnf**, añadiéndole la información contenida en el Apéndice A.4.

Ahora sólo queda crear la base de datos, usuarios, y las tablas necesarias.

8.4.2. Usuarios

Se han creado dos usuarios en MariaDB, cada uno con diferentes roles y accesos:

- **app**: Este usuario se conecta desde cualquier dirección IP (%) y es el usuario que usará la aplicación móvil, a través de Internet. El usuario se debe configurar con **REQUIRE TLS**, para obligar que use TLS.

- **guard**: Este usuario se conecta desde **localhost** y se encarga de insertar la información de cada evento (incidente) y la ruta del vídeo asociado en la base de datos. La conexión de

este usuario se desarrollará en el capítulo 10. También se debe configurar con **REQUIRE TLS**.

Debido a limitaciones técnicas y falta de información, no ha sido posible configurar mTLS en los usuarios de MariaDB. Por lo tanto, en vez de mTLS, se utilizará solo TLS para las conexiones de MariaDB, eso sí, garantizando que todas las comunicaciones entre el cliente y el servidor son seguras.

8.4.3. Base de datos y tablas

Se ha creado una base de datos llamada **PIOTGUARD_DB,** que contiene la tabla **events**. Esta tabla registra todos los incidentes relevantes detectados por el sistema. La estructura de la tabla es la siguiente:

- **id**: Campo **entero** que sirve como **clave primaria** y se autoincrementa con cada nuevo evento.
- **enabled**: Campo de tipo **tinyint** que indica si el sensor está activado o desactivado.
- **sensor_name**: Campo de tipo **varchar** que almacena el nombre del sensor que ha detectado el incidente (el que ha hecho disparar la alarma).
- **sensor_value**: Campo **entero** que indica el valor detectado por el sensor.
- **sensor_id**: Campo de tipo **smallint** que almacena el identificador del sensor.
- **date**: Campo de tipo **datetime** que registra la fecha y hora del incidente.
- **nombre_video**: Campo de tipo **varchar** que almacena la ruta del vídeo asociado al incidente.

8.5. Apache

Para configurar Apache con soporte SSL/TLS y mTLS, será necesario instalar los paquetes correspondientes de Apache y el módulo SSL. Los paquetes son: **apache2** y **libapache2-mod-ssl**.

La configuración de Apache se encuentra en la ruta **/etc/apache2**. Dentro de esta ruta, hay varias carpetas importantes: **ssl/**, donde se colocarán los certificados necesarios para el servidor de Apache, y **sites-available/**, donde se creará el archivo de configuración necesario.

Se deberá configurar Apache para que acepte conexiones desde Internet, especificar las rutas de los certificados para SSL/TLS, activar mTLS y definir la autenticación. Para esto, se modificará el archivo **default-ssl.conf**, añadiendo la información contenida en el Apéndice A.5.

En este archivo de configuración, se especifica que Apache utilizará el puerto 443 para conexiones desde Internet y se indican las ubicaciones de los certificados necesarios para establecer la conexión SSL/TLS. Además, se habilita mTLS y se define la configuración de verificación del cliente.

Los vídeos grabados por la cámara se guardarán en **/home/guard/records/**, y para que Apache pueda acceder directamente a ellos, se creará un enlace simbólico en **/var/www/html/videos**. También se especificarán los permisos y usuarios necesarios en el directorio para que Apache pueda acceder a ellos sin problemas.

Con esta configuración, se tiene un servidor Apache con mTLS implementado, y solo podrán acceder y reproducir los vídeos los clientes que presenten un certificado válido emitido por la CA en el servidor de Apache.

8.6. Servicios y systemd

Teniendo todos los servicios configurados, lo único que queda es activarlos y asegurarnos de que se reinicien en caso de caída y en cada inicio del sistema. Para Apache, Mosquitto y MariaDB, podemos utilizar directamente los siguientes comandos, y **systemd** se encargará de todo:

```
sudo systemctl enable apache2
sudo systemctl start apache2

sudo systemctl enable mosquitto
sudo systemctl start mosquitto

sudo systemctl enable mariadb
sudo systemctl start mariadb
```

Systemd es el sistema de gestión de servicios en Debian que se encarga de inicializar los servicios del sistema operativo, así como de supervisarlos y reiniciarlos en caso de fallo.

Sin embargo, para que todo funcione correctamente, también es necesario asegurarse de que, en caso de caída o fallo de los dispositivos virtuales y del módulo **v4l2loopback**, estos se reactiven automáticamente y se inicien al arrancar el sistema.

Para ello, se han creado dos servicios **videoloopback.service** y **modprobe-v4l2loopback.service**. El primero mantiene activo el envío de la fuente de vídeo a los dispositivos virtuales, y el segundo se encarga de cargar y mantener activo el módulo. Para que systemd se encarguen de ellos como los anteriores, simplemente escribimos en la terminal:

```
sudo systemctl enable videoloopback.service
sudo systemctl start videoloopback.service

sudo systemctl enable modprobe-v4l2loopback.service
sudo systemctl start modprobe-v4l2loopback.service
```

Además, es importante tener en cuenta los binarios y ejecutables resultantes de el **Capítulo** 10. Necesitamos que estos también actúen como los servicios anteriores. Más adelante (ver capítulo 10) se configurarán de la misma manera, creando servicios para que systemd se encargue de ellos. Esto asegurará que el sistema se mantenga siempre activo y monitorizando, cumpliendo con los requisitos de disponibilidad y fiabilidad necesarios del sistema.

8.7. NO-IP

Para permitir el acceso al servidor desde cualquier lugar a través de Internet, se necesitará configurar un servicio de **DNS dinámico (DDNS)**. Para ello se utilizará la herramienta NO-IP. NO-IP facilita la asignación de un nombre de dominio fijo a una dirección IP dinámica, es decir, que va cambiando. Con este nombre: **rzzz.ddns.net**, se podrá acceder al servidor desde cualquier punto, sin preocuparse por las variaciones de la IP pública.

El proceso de configuración de este servicio se hará siguiendo estos pasos:

1. **Registro en NO-IP**: Se deberá crear una cuenta en **NO-IP [33]**, y en este caso se registrará el nombre **rzzz.ddns.net**.

2. **Configuración del router**: Se configurará el router de manera que use el servicio DDNS de NO-IP, manteniendo la IP pública actualizada en caso de cambios.

Una vez hecho esto, ya podemos acceder a los servicios ofrecidos con el nombre de host **rzzz.ddns.net**.

Para tener el servidor completamente funcional, sólo faltaría desarrollar el cliente que corre dentro de la Raspberry Pi, el cual insertará la información de las incidencias y la ruta del vídeo en la base de datos, y el servidor de streaming, ofreciendo la captura de vídeo en tiempo real. Estos programas son descritos en el capítulo 10.

9

Programación del ESP32 y sensores

El programa desarrollado para el ESP32 está diseñado para manejar los sensores, monitorizar el entorno con ellos, y enviar su información al broker MQTT si se detectan anomalías y la alarma está activada.

El diseño del código se basa en el uso de **FreeRTOS**, permitiendo la ejecución concurrente de varias tareas en tiempo real, y aprovechando la **(NVS Non-Volatile Storage)**, una memoria persistente del ESP32, en la cual se guardarán las credenciales de configuración, contraseñas y y algunos datos, asegurando que no se pierdan entre reinicios del dispositivo, y garantizando su seguridad.

9.1. Modos de operación

Antes de flashear el código al ESP32, podremos configurar el sistema de dos formas; para almacenar datos, o leerlos de la NVS:

- **WRITE_MODE**: En este modo, se escriben las credenciales y configuraciones iniciales en la NVS. Esto incluye el **SSID** y la **contraseña** de la red WiFi, los **certificados** y las **claves** necesarias para la conexión mTLS con el broker MQTT, y las **credenciales de autenticación de Mosquitto**.

  Este modo, sólo será elegido si necesitamos actualizar o introducir datos en la NVS.

- **READ_MODE**: Este es el **modo operativo principal** del ESP32 una vez que las credenciales han sido almacenadas. El programa lee todas las credenciales y datos de la

NVS y las utiliza para establecer las conexiones necesarias, comenzar la monitorización de los sensores y ofrecer su funcionalidad al sistema.

9.2. FreeRTOS

FreeRTOS se utiliza en este diseño para crear y gestionar tareas concurrentemente. Las tareas diseñadas son:

- **DoorSensorTask**: Monitoriza el interruptor magnético que se colocará en puertas o ventanas. Detecta cambios en el estado del interruptor (abierto o cerrado). En caso de estar abierto, envía la información necesaria a la cola de datos si el sensor está activo.
- **PresenceSensorTask**: Monitoriza el sensor de presencia y envía los datos a la cola de datos cuando se detecta movimiento si el sensor está activo.
- **GasSensorTaks**: Lee los valores del sensor de gas a través del canal ADC configurado para este sensor (señal analógica), enviando los valores a la cola de datos periódicamente en caso de superar el nivel de gas o humo configurado si el sensor está activo.
- **mqttTask**: Esta ultima tarea maneja la conexión con el broker MQTT. Esta tarea es la que tiene la **prioridad más alta**, ya que necesitamos asegurar que los datos sean transmitidos y lleguen correctamente al servidor para que él los maneje adecuadamente. En esta tarea se autentica el cliente con el broker MQTT proporcionando sus credenciales y certificados, completando la conexión con mTLS. En caso de no conectarse, el dispositivo se reinicia.

Cola de datos

Una parte fundamental del programa es la cola de datos (**dataQ**), que se utiliza para la comunicación entre las tareas de los sensores y la **mqttTask**. Dado que las tareas de los sensores se ejecutan en paralelo, hay que tener un mecanismo para almacenar ordenadamente los datos leídos de los sensores.

Las tareas de los sensores van insertando datos en la cola si la alarma y los sensores están activos y detectan incidentes, y la función **publishValues** va extrayendo datos cada segundo y enviándolos al broker MQTT. Con esta implementación aseguramos que no se pierdan datos debido al paralelismo de las tareas, manejándolos de manera ordenada.

9.3. Publicación de topics. Función publishValues

Esta función se llama desde la **mqttTask**, y es la que extrae datos de la cola de datos **dataQ** y las va enviando al broker MQTT. Esta función sólo enviará datos si la alarma está activa.

9.4. Manejo de topics. Función topicHandler

Esta función se encarga de la gestión de las configuraciones de los sensores que envía la App Android a través del servidor. Cuando el ESP32 recibe el topic con la configuración de un sensor, esta función realiza lo siguiente:

1. **Analizar el JSON**: Dado que todo mensaje de MQTT en el sistema es de formato JSON, crea un objeto cJSON con los valores del mensaje.

2. **Identificar el topic**: Identifica el topic en el que se recibió el mensaje, y ejecuta su acción (por ejemplo, activar o desactivar la alarma, activar o desactivar el sensor especificado, etc.).

3. **Actualizar sensores**: Actualiza la configuración de los sensores y el sistema en la NVS, asegurando de que en caso de reinicio, el sensor mantenga el valor establecido (por ejemplo, si la alarma estaba desactivada y la App la activa, se asegura de que en caso de reinicio la alarma siga activada).

9.5. Configuración de los pines de los sensores

La configuración de los pines es lo primero en ejecutare en cada inicio del ESP32:

- **Sensor magnético**: Configurado en el GPIO4 del ESP32

- **Sensor de presencia**: Configurado en el GPIO13 del ESP32

- **Sensor de gas**: Configurado en el GPIO36; ADC1_CHANNEL_0 del ESP32, utiliza un canal ADC para leer los valores de concentración de gas o humo.

9.6. Otras funciones

- **Configuración de WiFi**: Al encenderse el ESP32, y una vez configurados los pines de los sensores y leída la NVS, configura el WiFi conectándose al SSID proporcionado con su contraseña. En caso de no obtener conexión, se reinicia.

- **Sincronización del Tiempo**: Para hacer que las conexiones mTLS funcionen correctamente, y para obetener la fecha y hora exacta cuando se detecta una anomalía, es necesario configurar la hora y fecha del ESP32. Esto se hace con la conexión a un servidor NTP (Network Time Protocol), que le proporciona la fecha y hora correcta. En caso de no obtenerla, se reinicia.

- **Publicaciones y suscripciones MQTT**: Una vez están todas las conexiones y configuraciones anteriores listas, el ESP32 se suscribe a los topics necesarios de MQTT.

10

Programación en la Raspberry Pi

Para desarrollar el cliente dentro del servidor (Raspberry Pi), y el **servidor RTSP**, necesitaremos integrar todos los servicios descritos en el capítulo 8 con varios programas desarrollados en C, obteniendo esta parte del sistema completamente funcional: haciendo stream correctamente, y grabando e insertando los datos necesarios cuando un incidente es detectado por parte del ESP32.

10.1. Servidor RTSP. GStreamer

Como ya sabemos, se utiliza GStreamer para el diseño del servidor RTSP. Este servidor enviará streaming al los clientes con SSL/TLS, donde la conexión se manejará directamente en el código fuente. Este programa ha sido diseñado basándose en **GStreamer Streaming Tutorial [10]** y **GStreamer RTSP SSL Example [11]**.

10.1.1. Configuración del Servidor RTSP

Al desarrollar el servidor RTSP con GStreamer, aseguramos que se pueda transmitir el vídeo eficientemente desde el dispositivo virtual **(/dev/video1)**. La configuración de este servidor se realiza mediante una **pipeline** de GStreamer, que procesa y transmite el vídeo en tiempo real.

La pipeline toma la entrada de vídeo desde el dispositivo, la convierte y la codifica en formato H.264 antes de transmitirla. La configuración también incluye la definición de los permisos y los perfiles de transmisión adecuados para asegurar la eficiencia en la transmisión:

```
gst_rtsp_media_factory_set_launch(factory, "( "
     "v4l2src device=/dev/video1 ! video/x- raw,format=I420,width=640,
   ↪ height=480,framerate=30/1 ! "
     "x264enc speed-preset=ultrafast ! h264parse config-interval=1 ! "
     "rtph264pay name=pay0 pt=96 "
     ")");
```

10.1.2. Autenticación y SSL/TLS

Para garantizar la seguridad de la conexión RTSP, se han configurado los certificados SSL/TLS y una base de datos de autenticación, proporcionada por la librería **gio.h**. La configuración incluye la carga de los certificados del servidor y de su CA. El servidor también maneja la autenticación de los clientes utilizando tokens y credenciales básicos de GStreamer, asegurando que solo los usuarios autorizados, si presentan los certificados correctos, puedan acceder a la transmisión de vídeo.

Función de callback accept_certificate

La función **accept_certificate** juega un papel crucial en este proceso, ya que verifica los certificados de los clientes, garantizando que sólo las conexiones autorizadas sean aceptadas, de manera que tanto el servidor como los clientes deben presentar certificados válidos para establecer la conexión.

10.1.3. Limpieza de Sesiones. Función timeout

Para mantener el servidor funcionando de manera eficiente, se implementa una función `timeout` que limpia periódicamente las sesiones expiradas. Esta función obtiene la **pool** de sesiones del servidor y elimina las que ya no están activas. Al realizar esta limpieza, se asegura que el servidor no se sobrecargue con sesiones inactivas, mejorando el rendimiento y asegurando que los recursos del sistema se utilicen de manera óptima.

10.1.4. Ejecución del Bucle Principal

El servidor se ejecuta continuamente mediante la ejecución del bucle principal de GStreamer (**g_main_loop_run**). Este bucle gestiona todas las conexiones de los clientes y transmite el vídeo en tiempo real si un cliente se conecta. Este bucle asegura que el servidor esté siempre disponible para transmitir vídeo cuando sea necesario, incluyendo:

- La inicialización del servidor.
- La gestión de los puntos de montaje donde los clientes se conectan para recibir el flujo de vídeo.
- La respuesta a las solicitudes de conexión y transmisión de los clientes.

Como se comentó al final de el capítulo 8, a este programa también se ha hecho servicio, configurándolo con systemd al igual que los anteriores, para asegurar que el servidor esté siempre activo.

10.2. Cliente servidor

Para gestionar la lógica del sistema de vigilancia en el servidor, se desarrolla un programa en C que utiliza varias bibliotecas como **mosquitto** para la comunicación MQTT, **mysql** para la interacción con MariaDB y **cJSON** para manejar los mensajes recibidos de los topics de MQTT, entre otras. Las principales funciones de este programa son:

10.2.1. Conexión y Configuración del cliente MariaDB

La función **mariadb_init** se encarga de inicializar y conectarse con la base de datos. Utiliza los certificados SSL/TLS, garantizando la seguridad en la conexión, y además autentica este cliente con el servidor proporcionando sus credenciales.

Inserción de Datos. Función insertEvent

La función **insertEvent** inserta un nuevo evento (incidente) en la tabla **events** cuando se detecta un incidente, incluyendo detalles como la fecha, el nombre del sensor, el valor detectado

y la ruta del vídeo grabado. Utiliza sentencias preparadas de MySQL para asegurar la integridad y seguridad de los datos.

10.2.2. Grabación de Vídeo. Función record_video

La función **record_video** inicia la grabación de vídeo cuando se detecta un incidente. Construye un comando del sistema que ejecuta un **script bash**, el cual usa GStreamer para realizar la grabación. El nombre del archivo de vídeo se genera dinámicamente basándose en la fecha y hora y el sensor que lo detectó.

Actualmente, los videos almacenados no se cifran, pero como trabajo futuro, sería ideal implementar el cifrado de los videos para asegurar que los contenidos sean inaccesibles sin una clave de cifrado adecuada.

Conexión y Configuración del cliente MQTT. Función mqtt_init

La función **mqtt_init** se encarga de inicializar la biblioteca **mosquitto** y configurar las opciones SSL/TLS y mTLS para la comunicación con el broker MQTT. Esto incluye especificar las rutas a los certificados del cliente y la CA, y configurar la autenticación para la conexión. También se suscribe a los topics necesarios, y los **callbacks** con la función **mqttCallback** manejando los mensajes recibidos.

Manejo de Mensajes MQTT

El callback `mqttCallback` maneja los mensajes que llegan a los temas suscritos:

- Para el topic **ALARM_TRIGGER**, topic de notificación de alarma, el mensaje contiene datos en JSON sobre el evento detectado. La función:
  - Analiza el JSON para extraer los detalles del evento.
  - Llama a **record_video** para iniciar la grabación.
  - Inserta los detalles del evento extraídos del JSON en la base de datos con **insertEvent**.

Bucle Principal de MQTT. Función mqtt_loop

La función **mqtt_loop** mantiene un bucle infinito que gestiona la conexión con el broker MQTT y la recepción de mensajes. Esto asegura que el cliente esté siempre activo y listo para procesar eventos en tiempo real.

10.2.3. Función Principal, main

La función principal (**main**) del programa se encarga de:

- Inicializar las conexiones MQTT y MariaDB.
- Ejecutar **mqtt_loop** para procesar mensajes.

11

Programación de la App Android

11.1. Estructura general

La estructura general de la aplicación se basa en la utilización de **Activities** para manejar las diferentes pantallas de la aplicación, y **Services** para gestionar las tareas en segundo plano, como la conexión con el broker MQTT.

11.1.1. Actividades (Activities)

En Android, las **Activities** son componentes que proporcionan una **IU** (Interfaz de Usuario). Cada Activity se asocia con un archivo **layout XML** que define la estructura visual. Por ejemplo, **MainActivity** se asocia con **main_layout.xml**. En la Activity se gestiona toda la lógica de esa vista, mientras que lo que ve el usuario es el layout. Las Activities permiten a los usuarios interactuar con la aplicación, proporcionando botones, textos y otros elementos de IU.

11.1.2. Servicios (Services)

Los **servicios** se utilizan para gestionar tareas en segundo plano. En esta aplicación, **MqttService** es un servicio que mantiene la conexión con el broker MQTT y gestiona la suscripción de topics y publicación de mensajes.

11.2. Funcionalidades Principales

11.2.1. Gestión del Sistema y Sensores

La gestión del sistema y los sensores se realiza a través de la **MainActivity** y el **MqttService**.

Actividad Principal MainActivity

La **MainActivity** es la actividad principal de la App, donde el usuario puede interactuar con los diferentes sensores y funciones del sistema. Desde esta actividad, el usuario puede activar y desactivar sensores, ir al reproductor de streaming y navegar a la vista de **Event History**, donde se muestran los indecentes detectados en forma de lista, recuperados de la base de datos del servidor.

Servicio MqttService

El **MqttService** es el encargado de gestionar la conexión con el broker MQTT utilizando mTLS. Este servicio permite que la aplicación se comunique de manera bidireccional con el servidor, publicando mensajes y suscribiéndose a los topics necesarios para recibir actualizaciones. Utiliza certificados almacenados en un archivo **.p12** para configurar la conexión mTLS, utilizando un **SecureSocket**, garantizando la integridad y seguridad de los datos transmitidos.

Al diseñar la conexión MQTT como un servicio, aseguramos que la aplicación pueda recibir actualizaciones y enviar comandos para los sensores sin necesidad de estar en primer plano. Esto garantiza una comunicación constante manteniendo al usuario informado, permitiéndole interactuar con el sistema de seguridad en cualquier momento.

11.2.2. Streaming y GStreamer

Será necesario desarrollar un cliente RTSP con GStreamer para poder ver el streaming ofrecido por el servidor RTSP correctamente. Este cliente gestionará la conexión al servidor RTSP, decodificará el flujo de vídeo y lo renderizará en la vista correspondiente, pudiéndolo reproducir y ver el vídeo.

StreamActivity

La Activity **StreamActivity** es la responsable de gestionar la interfaz de usuario para la transmisión de vídeo, proporcionando un reproductor en el cual se verá el streaming. Esta Activity configura y controla la vista donde se renderiza el vídeo transmitido desde el servidor RTSP.

En esta Activity es donde se proporcionan todos los certificados y credenciales necesarios para realizar la conexión al servidor RTSP con SSL/TLS. Estos certificados y credenciales son enviados al código C a través de **JNI (Java Native Interface)**, que maneja la lógica del cliente RTSP en la App. **JNI** es una interfaz de programación que permite a las aplicaciones Java llamar funciones y utilizar bibliotecas escritas en otros lenguajes de programación, que en nuestro caso, es C.

GStreamerManager

La clase **GStreamerManager** es un **singleton** que gestiona la biblioteca de GStreamer, asegurando que solo se inicialice una vez. Un **Singleton** en Android es un patrón de diseño que asegura que una clase tenga **solo una instancia en todo el ciclo de vida** de la aplicación. Esto es crucial para evitar conflictos y errores que puedan surgir de múltiples inicializaciones de la biblioteca de GStreamer.

Los **métodos nativos**, que son son funciones en Java que se implementan en un lenguaje de programación distinto (en este caso C), se llaman desde Kotlin para iniciar, pausar y detener la transmisión de vídeo. Estos métodos interactúan con la lógica implementada en C, manejando directamente la pipeline de video.

GStreamerManager se encarga de todas las interacciones con GStreamer, incluyendo la configuración de la pipeline, la gestión de la superficie de vídeo y estado de la transmisión, centralizando la lógica de GStreamer de Java en una sola clase.

Lógica en C

La lógica del cliente RTSP de GStreamer se implementa en C, gestionando la conexión SSL/TLS y creando la pipeline de vídeo, configurándola para recibir y decodificar el flujo de vídeo RTSP, utilizando los certificados y credenciales proporcionados por la **StreamActivity**. Se utilizan métodos nativos para comunicar a la **StreamActivity** el estado de la pipeline.

Similar al código del **Server RTSP** en la Raspberry Pi, esta implementación en C incluye

la creación de la pipeline de GStreamer con los elementos necesarios, como la decodificación, y el renderizado de vídeo en la vista de la App.

Esta integración entre Java, Kotlin y C a través de JNI permite aprovechar la robustez y flexibilidad de GStreamer para la transmisión de vídeo en una aplicación Android.

11.2.3. Gestión de Credenciales

Para proteger las credenciales, evitando que estén directamente escritas en el código fuente, se utiliza **SecureStorage**.

SecureStorage

SecureStorage es una clase diseñada para gestionar el almacenamiento de credenciales y datos en la App Android. Utiliza el **Keystore**, un sistema que permite almacenar y gestionar claves de forma segura, proporcionando una capa adicional de seguridad y evitando que estén expuestas en el código fuente.

La clase SecureStorage proporciona:

- **Almacenamiento**: El Keystore de Android se emplea para generar y gestionar claves de cifrado. **SecureStorage** utiliza estas claves para cifrar los datos sensibles antes de almacenarlos en una estructura llamada **SharedPreferences**. Incluso si alguien accede a los datos almacenados, no podrá entender la información sin las claves de cifrado adecuadas.

- **Cifrado y Descifrado**: **SecureStorage** proporciona métodos para cifrar y descifrar datos. Al guardar datos sensibles, como las credenciales de usuario, los datos se cifran utilizando una clave almacenada en el Keystore. Cuando se necesitan estos datos, **SecureStorage** los descifra para su uso. Este proceso asegura que las credenciales estén siempre protegidas mientras están almacenadas y solo se descifran cuando se necesitan.

Este enfoque asegura que las credenciales y otros datos sensibles nunca se expongan directamente en el código fuente, aumentando la seguridad de la aplicación. Sin embargo, actualmente las credenciales se introducen además mediante la clase **StoreCredentials**, que las almacena de manera segura usando **SecureStorage**.

En el futuro, se planea mejorar este proceso para que el cifrado y la gestión de credenciales sean seguros, no teniendo que introducir manualmente las credenciales en el código fuente. Esto incluirá la implementación de prácticas más avanzadas de gestión de claves y credenciales, mejorando aún más la seguridad de la App Android y del sistema.

11.2.4. Historial de Eventos y Grabaciones de Vídeo

La Activity que nos muestra el historial de eventos, es decir, el registro de las anomalías detectadas, es la **DbActivity** junto a la **RecyclerView**.

La **DbActivity** se encarga de realizar la conexión con la base de datos **MariaDB** utilizando SSL/TLS en la conexión. Al iniciar la actividad, se cargan los eventos desde la base de datos y se muestran en la **RecyclerView**. La actividad también proporciona la funcionalidad de eliminar eventos, actualizando la interfaz de usuario.

Para gestionar la conexión SSL/TLS, se utiliza un archivo **.p12** que contiene los certificados necesarios. Estos se cargan y se utilizan para crear un **SecureSocket** que garantiza la privacidad e integridad de los datos transmitidos entre la aplicación y la base de datos.

La **RecyclerView** en la **DbActivity** se utiliza para mostrar la lista de eventos recuperados de la base de datos. Cada elemento del **RecyclerView** representa un evento registrado, mostrando información relevante como el **ID del sensor**, el **nombre del sensor**, el **valor detectado**, la **fecha del evento** y, si está disponible, la **ruta al vídeo asociado**.

Cada elemento del **RecyclerView** incluye un botón para **eliminar el evento** correspondiente de la base de datos. Al hacer clic en este botón, se realiza una solicitud para eliminar el evento, y una vez confirmada la eliminación, se actualiza la interfaz de usuario para reflejar este cambio.

Reproducción de Vídeos

Como cada evento tiene su grabación asociada, cada ítem del **RecyclerView** (evento) contiene también un botón permitiendo reproducir dicho vídeo. Al hacer clic en el botón, se inicia una nueva actividad, la **PlayerActivity**, que se encarga de reproducir el vídeo almacenado en el servidor gracias a Apache. Esto se realiza a través de una conexión HTTPS utilizando mTLS, garantizando que el acceso al contenido multimedia sea seguro.

PlayerActivity y Apache

La **PlayerActivity** es la responsable de gestionar la reproducción de vídeo asociado a cada evento. Utiliza un layout dedicado para mostrar el reproductor de vídeo, permitiendo a los usuarios ver las grabaciones de los incidentes. La **PlayerActivity** también se encarga de realizar la conexión mTLS con el servidor de Apache, y como los demás clientes, necesita un archivo **.p12** para los certificados y creando un **SecureSocket** para la conexión, accediendo al vídeo y mostrándolo en el reproductor de vídeo.

12

Resultados y Líneas Futuras

12.1. Resultados

El desarrollo de este Proyecto de Fin de Grado ha finalizado con la creación de un sistema de seguridad que combina diversas tecnologías, servicios y seguridad, ofreciendo una solución muy robusta y segura. A lo largo de este documento, se han descrito varias funcionalidades clave:

- **Monitorización continua**: El ESP32 monitoriza constantemente el entorno donde se instale mediante sus sensores, enviando mensajes de alarma en caso de detección de alguna anomalía.
- **Streaming de Vídeo**: La Raspberry Pi con su cámara ofrece streaming de vídeo en tiempo real, permitiendo ver el vídeo en la App Android, y a parte grabar vídeo. Gracias los LEDs infrarrojos de la cámara, obtenemos imagen en condiciones de poca luz.
- **Gestión de Eventos**: En la App Android podremos ver y gestionar los incidentes detectados por el sistema (eventos), pudiendo ver las grabaciones e información sobre los mismos.
- **Notificaciones**: La App Android recibirá notificaciones en caso de que el sistema detecte una anomalía, avisando al usuario incluso estando la App en segundo plano.
- **SSL/TLS y mTLS**: Se utiliza SSL/TLS para proteger todas las conexiones y comunicaciones, y mTLS en servicios como Apache y MQTT, garantizando cifrado y autenticación mutua para mayor seguridad.

12.2. Conclusiones

Se ha logrado desarrollar un sistema de seguridad proporcionando una vigilancia constante, integrando herramientas y tecnologías haciendo al sistema completo funcional. La implementación de medidas de criptografía y seguridad en las conexiones asegura la protección de los datos transmitidos, manteniendo la integridad y confidencialidad de la información. Además, se han establecido mecanismos para garantizar que el sistema permanezca siempre activo, incluso ante posibles fallos, asegurando así una vigilancia ininterrumpida. Estas características hacen que el sistema sea una solución integral y robusta para la seguridad del entorno monitorizado.

- **Integración de Herramientas**: El uso e integración de varias tecnologías tanto hardware como software ha permitido que todo el sistema funcione como si se tratase de una única entidad: desde la monitorización del entorno con el ESP32, pasando por la gestión de esa información y el streaming de video, hasta el control y visualización de toda esta información en la App Android. Todas las partes trabajan juntas y se necesitan unas a otras para ofrecer un sistema de completo y funcional.

- **Seguridad**: Se ha puesto especial énfasis en la seguridad de las comunicaciones dentro del sistema. Utilizando SSL/TLS y mTLS, aseguramos que todas las conexiones estén protegidas y cifradas. Como ya se ha descrito, la autenticación mutua garantiza que tanto el servidor como los clientes se verifiquen entre sí, protegiendo la integridad y confidencialidad de los datos transmitidos.

- **Siempre activo**: Se han implementado medidas para que todos los servicios del sistema estén siempre activos y funcionando, incluso en caso de fallos. Esto incluye el mecanismo de reconexión automática para el cliente MQTT, reinicios de todos los servicios del servidor, y reinicio del ESP32. Estas medidas aseguran que el sistema se mantenga operativo en todo momento, proporcionando una monitorización y vigilancia constante.

12.3. Trabajos Futuros

A pesar de todos los objetivos conseguidos, hay varias áreas en las que se pueden realizar mejoras, ya sea aumentando las funcionalidades y rendimiento, o en seguridad del sistema al completo. A lo largo del documento ya se han comentado varias mejoras, como puede ser el **Login**, y la gestión de credenciales **SecureStorage** en la App Android. Es este capítulo profundizamos más sobre las posibles mejoras:

- **Implementar el Login con MariaDB**: Desarrollar la funcionalidad de login y gestión de usuarios utilizando MariaDB, permitiendo gestionar los usuarios, modificar contraseñas e incluso crear nuevas cuentas.
- **Pruebas y Depuración**: Realizar más pruebas para identificar y corregir posibles errores, además de depurar el código desarrollado, aumentando una mayor estabilidad del sistema.
- **Sensibilidad del Sensor de Gas**: Implementar la capacidad de configurar el límite de detección de gas directamente desde la App Android, permitiendo ajustar la sensibilidad del sensor.
- **Keyring en Raspberry Pi**: **Keyring** es una herramienta que permite guardar y gestionar contraseñas y otros datos sensibles de manera segura. Utilizar Keyring en la Raspberry Pi permitirá gestionar y almacenar credenciales de manera más segura, mejorando la protección de estos datos.
- **Mejorar SecureStorage**: Implementar y desarrollar una versión más segura de SecureStorage en la App Android, manteniendo las credenciales y datos protegidos.
- **mTLS para MariaDB y Servidor RTSP**: Implementar mTLS no solo para la comunicación MQTT, sino también para las conexiones con MariaDB y el servidor RTSP, aumentando la seguridad de todas las comunicaciones del sistema.
- **Hardware de Alarma e Información**: Integrar un hardware de alarma que pueda sonar en caso de detección de anomalías, alertando físicamente en el entorno monitorizado. Además, incorporar botones físicos para activar y desactivar la alarma, junto con una pantalla que muestre información relevante del sistema.

- **Actualizar Algoritmos de Cifrado**: Reemplazar algoritmos anticuados como RSA y DES3 por opciones más actuales y seguras, como Ed25519 y AES, respectivamente. Estos algoritmos ofrecen una mayor seguridad y eficiencia, protegiendo mejor los datos transmitidos.

- **Mejorar el Proceso de SSL/TLS**: Actualmente, los certificados SSL/TLS utilizados en el sistema no son los más seguros posibles. Es importante incluir extensiones en los certificados que indiquen explícitamente su uso.

- **Cifrado de Certificados del Servidor**: Para mejorar la seguridad del sistema, es esencial que las claves privadas del servidor estén cifradas. Utilizar herramientas como **Keyring** para manejar estas claves cifradas puede proporcionar una capa adicional de seguridad.

- **Cifrado de Vídeo**: Implementar el cifrado de las grabaciones antes de su almacenamiento para garantizar que los contenidos sean inaccesibles sin la clave de cifrado adecuada.

Estas mejoras y futuras implementaciones ayudarán a perfeccionar el sistema de seguridad, haciéndolo más seguro y adaptable.

13

Referencias Bibliográficas

[1] **Raspberry Pi Operating Systems**:
https://www.raspberrypi.com/software/operating-systems/

[2] **Mosquitto API Documentation**:
https://mosquitto.org/api/files/mosquitto-h.html

[3] **Eclipse Paho Android Client**:
https://eclipse.dev/paho/index.php?page=clients/android/index.php

[4] **Eclipse Paho Android Javadoc**:
https://eclipse.dev/paho/files/android-javadoc/index.html

[5] **GStreamer Linux Installation**:
https://gstreamer.freedesktop.org/documentation/installing/on-linux.html?gi-language=c

[6] **GStreamer Android Development**:
https://gstreamer.freedesktop.org/documentation/installing/for-android-development.html?gi-language=c

[7] **GStreamer Basic Tutorials**:
https://gstreamer.freedesktop.org/documentation/tutorials/basic/index.html?gi-language=c

[8] **GStreamer Android Tutorials**:
https://gstreamer.freedesktop.org/documentation/tutorials/android/index.html?gi-language=c

[9] **GStreamer RTSP Server**:
https://gstreamer.freedesktop.org/documentation/gst-rtsp-server/index.html?gi-language=c

[10] **GStreamer Streaming Tutorial**:
https://gstreamer.freedesktop.org/documentation/tutorials/basic/streaming.html?gi-language=c

[11] **GStreamer RTSP SSL Example**:
https://github.com/enthusiasticgeek/gstreamer-rtsp-ssl-example

[12] **MariaDB Documentation**:
https://mariadb.com/kb/en/documentation/

[13] **MariaDB Securing Connections**:
https://mariadb.com/kb/en/securing-connections-for-client-and-server/

[14] **MariaDB Connector/J**:
https://mariadb.com/kb/en/about-mariadb-connector-j/

[15] **MariaDB C Connector**:
https://mariadb.com/docs/server/connect/programming-languages/c/install/

[16] **Apache HTTP Server Documentation**:
https://httpd.apache.org/docs/current/

[17] **Apache SSL How-To**:
https://httpd.apache.org/docs/2.4/ssl/ssl_howto.html

[18] **Android Activity Reference**:
https://developer.android.com/reference/android/app/Activity

[19] **Android Services Guide**:
https://developer.android.com/develop/background-work/services?hl=es-419

[20] **Android Security Config**:
https://developer.android.com/privacy-and-security/security-config?hl=es-419#CustomTrust

[21] **Bouncy Castle JavaDoc**:
https://downloads.bouncycastle.org/java/docs/bcprov-jdk18on-javadoc/

[22] **Android Notification Reference**:
https://developer.android.com/reference/android/app/Notification

[23] **ESP-IDF MQTT API Reference**:
https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/protocols/mqtt.html

[24] **ESP-IDF Mutual Auth Example**:
https://github.com/espressif/esp-idf/tree/v5.2.1/examples/protocols/mqtt/ssl_mutual_auth

[25] **ESP-IDF Wi-Fi API**:
https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/network/esp_wifi.html

[26] **ESP-IDF NVS Flash**:

https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/storage/nvs_flash.html

[27] **ESP-IDF TLS API**:

https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/protocols/esp_tls.html

[28] **FreeRTOS Documentation**:

https://www.freertos.org/a00106.html

[29] **cJSON GitHub Repository**:

https://github.com/DaveGamble/cJSON

[30] **ESP32 DevKitC Getting Started**:

https://docs.espressif.com/projects/esp-idf/en/latest/esp32/hw-reference/esp32/get-started-devkitc.html

[31] **PIR Sensor User Manual**:

https://puntoflotante.net/MANUAL-DEL-USUARIO-SENSOR-DE-MOVIMIENTO-PIR-HC-SR501.pdf

[32] **MQ-2 Gas Sensor Documentation**:

https://docs.sunfounder.com/projects/umsk/en/latest/01_components_basic/04-component_gas.html

[33] **NO-IP**:

https://www.noip.com/

Apéndice A
Contenido de archivos

A.1. Archivo de configuración openssl.cnf

```
[ req ]
prompt = no
default_bits = 4096
distinguished_name = req_distinguished_name
req_extensions = req_ext
[ req_distinguished_name ]
C = ES
ST = Malaga
L = Malaga
O = PioTGuard
OU = Nombre del Servicio (por ejemplo MQTT)
CN = rzzz.ddns.net
emailAddress = ejemplo@outlook.com

[ req_ext ]
subjectAltName = @alt_names

[ alt_names ]
DNS.1 = localhost
DNS.2 = rzzz.ddns.net
IP.1 = 192.168.1.211

[ v3_ca ]
subjectAltName = @alt_names
```

A.2. Archivo de configuración Mosquitto conf.cnf

```
# Mosquitto Configuration
# Secure connections
listener 8883 0.0.0.0 # Internet
cafile /etc/mosquitto/certs/mqtt_ca.crt
certfile /etc/mosquitto/certs/mqtt_server.crt
keyfile /etc/mosquitto/certs/mqtt_server.key
# Enable mutual TLS
require_certificate true

# Password file for client authentication
password_file /etc/mosquitto/passwd
```

A.3. Archivo de configuración MariaDB 50-server.cnf

```
    [server]

    [mysqld]
    max_allowed_packet=16M
    wait_timeout=28800
    connect_timeout=10

    bind-address            = 0.0.0.0

    ssl-ca = /etc/mysql/ssl/mariadb_ca.crt
    ssl-cert = /etc/mysql/ssl/server.crt
    ssl-key = /etc/mysql/ssl/server.key
    require-secure-transport = on

    [embedded]

    [mariadb]

    [mariadb-10.5]
```

A.4. Archivo de configuración MariaDB 50-clients.cnf

```
# This group is read by the client library
# Use it for options that affect all clients, but not the server

[client]
# Example of client certificate usage
ssl-ca = /etc/mysql/ssl/mariadb_ca.crt
ssl-cert = /etc/mysql/ssl/db_client.crt
ssl-key = /etc/mysql/ssl/db_client.key

# Allow only TLS encrypted connections
ssl-verify-server-cert = on
[client-mariadb]
```

A.5. Archivo de configuración Apache default-ssl.cnf

```
    <IfModule mod_ssl.c>
  <VirtualHost _default_:443>
    SSLEngine on #SSL/TLS

    SSLCertificateFile  /etc/apache2/ssl/server.crt
    SSLCertificateKeyFile /etc/apache2/ssl/server.key
    SSLCACertificateFile /etc/apache2/ssl/apache_ca.crt

    SSLVerifyClient require #mTLS
    SSLVerifyDepth  10
    SSLProtocol all -SSLv3 -TLSv1 +TLSv1.2

    <Directory "/var/www/html/videos">
            SSLRequireSSL
            SSLVerifyClient require
            SSLVerifyDepth  10
            SSLOptions +StdEnvVars
            Options FollowSymLinks
            Require all granted
      </Directory>

    <FilesMatch "\.(cgi|shtml|phtml|php)$">
        SSLOptions +StdEnvVars
    </FilesMatch>
    <Directory /usr/lib/cgi-bin>
        SSLOptions +StdEnvVars
    </Directory>
  </VirtualHost>
</IfModule>
```



UNIVERSIDAD DE MÁLAGA | uma.es

E.T.S de Ingeniería Informática
Bulevar Louis Pasteur, 35
Campus de Teatinos
29071 Málaga

E.T.S. DE INGENIERÍA INFORMÁTICA